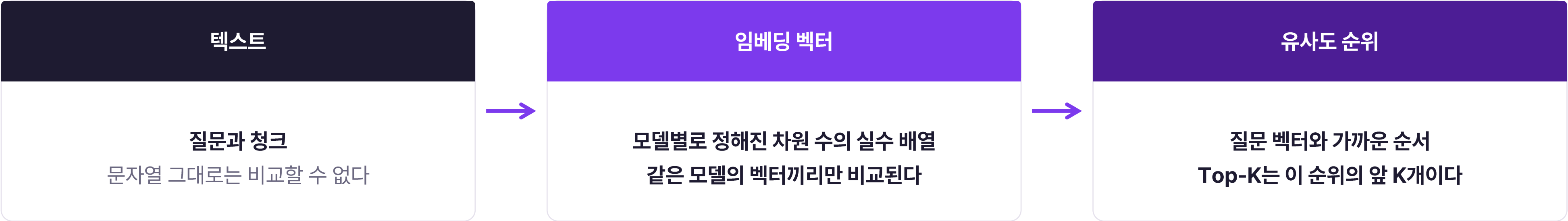


Retrieval Metrics: 검색 성능 측정

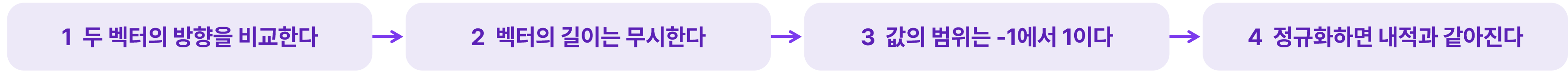


임베딩과 벡터 검색의 원리

검색은 의미가 비슷한 벡터를 찾는 계산이다. 유사도가 어떻게 계산되는지가 중요하다.



코사인 유사도의 계산 기준



질문과 청크를 같은 모델로 임베딩해야 비교가 성립한다. 이 순위를 채점하는 것이 검색 지표이다.



검색 채점의 구조

평가셋 한 문항이 채점되는 전 과정이다.
정답 ID는 미리 라벨해 둔 것으로 보고, 오른쪽 지표는 모두 이 예시에서 계산한 값이다.

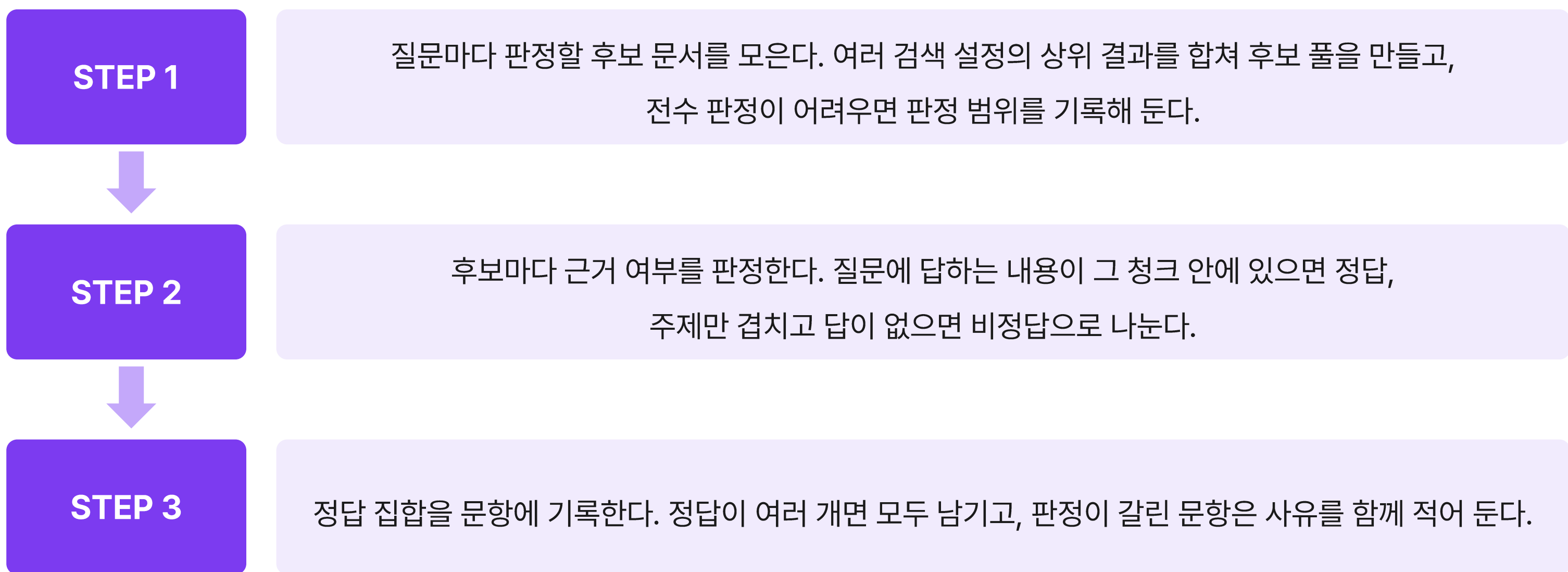


LLM 호출이 없는 채점이다. 평가셋(질문·정답 ID)만 있으면 검색 품질은 집합·순위 계산으로 끝난다.



검색 평가셋: 정답 집합 만들기

지표는 정답 집합을 전제로 계산된다. 그 집합을 어떻게 만들었는지가 점수의 신뢰도에 크게 작용한다.



정답 집합에 없는 문서를 검색기가 반환하면 오답으로 계산된다. 라벨에서 빠진 근거는 어떤 검색기가 반환해도 점수로 인정되지 않는다.



분류 문제로 보는 검색 평가

검색된 문서를 "연관 있다 / 없다"로 판정하면, 검색 평가는 이진 분류 문제가 된다.



Precision · 얼마나 정확하게 검색했는가

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

검색된 문서 중 연관 문서의 비율이다. K를 키우면 대체로 떨어진다.

Recall · 얼마나 빠짐없이 검색했는가

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

전체 연관 문서 중 검색된 문서의 비율이다. RAG 검색의 1차 관문이다.

Accuracy를 사용하지 않는 이유

코퍼스 대부분은 무관 문서(TN)이다. 아무것도 검색하지 않아도 Accuracy는 높게 나오기 때문에, 검색 평가에서는 판별력이 없다.

Top-K 검색에는 두 지표의 @K 버전(Precision@K · Recall@K)을 사용한다. 이 관점이 검색 지표의 출발점이다.

Precision-Recall이 놓치는 두 가지

Top-10에 정답 5개. 지표가 같은 두 검색 결과의 품질이 같은지 확인한다.

검색 결과 A

1 · 정답
2 · 정답
3 · 정답
4 · 정답
5 · 정답
6
7
8
9
10
P@10 0.5 · R 5/8

검색 결과 B

1
2
3
4
5
6 · 정답
7 · 정답
8 · 정답
9 · 정답
10 · 정답
P@10 0.5 · R 5/8

=

한계 1 · 정답이 여러 개다

분류 관점은 문서를 하나씩 독립적으로 판정한다. 한 질문에 근거 청크가 여럿이고, 그중 무엇을 어디에 놓았는지가 품질에 작용한다는 점을 반영하지 못한다.

한계 2 · 순위 미반영

컨텍스트는 위에서부터 채워지고, LLM도 위쪽 근거부터 활용한다. 정답 순위가 낮아져도 Precision·Recall은 변하지 않는다.

따라서 순위를 반영하는 지표가 필요하다

MAP · MRR · NDCG

지표는 동일하지만, A는 상위 순위의 근거로 답하고 B는 하위 순위의 근거로 답한다.



Retrieval Metrics 용어 정리

같은 Top-K 결과라도 지표마다 측정하는 관점이 다르므로 용어 정의부터 맞춘다.

Hit Rate@K

Top-K 안에 정답이 하나라도 있으면 1, 없으면 0.
가장 관대한 지표이다.

Precision@K

Top-K 중 정답의 비율.
K를 키우면 대체로 떨어진다.

Recall@K

전체 정답 중 Top-K에 들어온 비율.
RAG 검색의 1차 관문이다.

MRR

첫 정답 순위 r 의 역수 $1/r$ 평균.
정답이 몇 위에 나오는지에 민감하다.

NDCG

순위 가중 이득을 이상적 순위와 비교해
정규화한 지표. 순서 품질까지 본다.

Context P/R

RAGAS의 동명 지표는 LLM이 관련성·주장을 판정
하는 방식이다. 정의가 다르므로 혼용하지 않는다.



MAP: 순위를 반영하는 평균 정밀도

관련 문서가 나온 순위에서의 precision@k만 모아 평균한 값이다. 적중 수가 같아도 순서가 좋으면 값이 커진다.

① Precision@k
(k번째까지의 관련 수) / k
순위 k까지의 정밀도



② AP · 평균 정밀도
 $(1/R) \sum P@k \cdot rel_k$
관련 문서가 나온 순위의 P@k만 더해 정답 수 R로 나눈다



③ MAP
 $(1/|Q|) \sum AP(q)$
문항별 AP의 평균 · 평가셋 전체 점수

같은 3개 적중, 다른 순서
위쪽에 몰릴수록 AP가 커진다.

✓✓x → 1.00

✓x✓ → 0.76

x✓✓ → 0.48

k	rel _k	P@k	AP 기여
1	1	1 / 1 = 1.000	1.000
2	0	1 / 2 = 0.500	-
3	1	2 / 3 = 0.667	0.667
4	0	2 / 4 = 0.500	-
5	1	3 / 5 = 0.600	0.600
합계	R = 3	기여 합 2.267	AP = 0.756

관련도 [1, 0, 1, 0, 1]인 검색 결과의 AP 계산 (정답 3개)

정답이 하나뿐인 문항에서는 AP가 RR과 같아지고, MAP은 MRR과 같아진다. 분모는 정답 총 개수를 쓰는 것이 표준 정의이다.



이진 관련도와 등급 관련도

Hit와 MRR도 관련도를 0과 1로만 본다. 근거의 유용성까지 측정하려면 등급이 필요하다.

이진 관련도 · 지금까지

1 · 연관 있다

0 · 무관하다

정답 여부만 본다.

Hit은 존재 여부, MRR은 첫 정답의 위치만 반영한다. 두 번째, 세 번째 근거의 위치는 두 지표에 반영되지 않는다.

등급 관련도 · 지금부터

3 · 질문에 바로 답하는 근거

2 · 부분 근거

1 · 배경 설명

0 · 무관

같은 "연관"이라도 유용성이 다르다.

바로 답하는 청크와 배경 정보만 제공하는 청크를 구분해야 "좋은 문서가 상위에 있는가"를 측정할 수 있다.

가정 ① 더 관련된 문서가 더 유용하다
등급이 높은 문서가 위에 올수록 이득이 크다

가정 ② 아래 순위일수록 읽힐 가능성이 낮다
같은 문서라도 하위 순위에 있으면 가치를 할인한다



이 두 가정을 수식으로 옮긴 지표가 DCG(Discounted Cumulative Gain)이다.



DCG의 계산: 이득 누적과 순위 할인

상위 순위의 관련도 이득(gain)은 그대로 더하고, 순위가 내려갈수록 $\log_2(i+1)$ 로 나누어 이득을 줄인다.

① Gain (이득)

rel_i

i번째 결과의 관련도 등급



② CG@K · 누적

$\sum rel_i$

이득의 단순 합 · 순서를 반영하지 못한다



③ DCG@K · 할인 누적

$\sum rel_i / \log_2(i+1)$

순위가 뒤로 갈수록 이득을 줄여서 더한다

순위별 할인 배율

상위에 놓인 근거일수록 이득이 그대로 반영된다.

1위 $\times 1.00$

2위 $\times 0.63$

3위 $\times 0.50$

4위 $\times 0.43$

9위 $\times 0.30$

k	rel_i	$rel_i / \log_2(i+1)$	DCG@k
1	3	3.00	3.00
2	2	1.26	4.26
3	3	1.50	5.76
4	0	0.00	5.76
5	0	0.00	5.76
6	1	0.36	6.12
7	2	0.67	6.78
8	2	0.63	7.42
9	3	0.90	8.32
10	0	0.00	8.32

관련도 [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]인 검색 결과의 누적 과정이다. 0인 자리에서는 점수가 늘지 않는다.

Recall이 "검색했는가"를 본다면, DCG는 "얼마나 좋은 순서로 검색했는가"를 본다.
다만 DCG는 상한이 없는 값이므로 문항끼리 곧바로 비교하기 어렵다.

NDCG의 계산: 이상적 순위 대비 정규화

DCG는 상한이 없다. 문항마다 가능한 최댓값(IDC@K)으로 나눠야 문항끼리 비교할 수 있다.

$$\text{NDCG@K} = \text{DCG@K} / \text{IDCG@K}$$

IDCG@K는 관련도를 내림차순으로 정렬했을 때의 DCG,
즉 그 문항에서 나올 수 있는 최대 점수이다.

값의 범위 0 ~ 1

어떤 문항이든 이상적 순위와의 비율이므로 1을 넘지 않는다.

1.0 = 그 문항의 상한

점수가 10이면 관련도 순서대로 완벽하게 배치한 것이다.

문항 간 평균 가능

정답 수가 4개든 8개든 동일한 기준으로 측정한 값이므로 평가셋 평균을 낼 수 있다.

k	rel _i	DCG@k	이상 rel _i	IDCG@k	NDCG@k
1	3	3.00	3	3.00	1.00
2	2	4.26	3	4.89	0.87
3	3	5.76	3	6.39	0.90
4	0	5.76	2	7.25	0.79
5	0	5.76	2	8.03	0.72
6	1	6.12	2	8.74	0.70
7	2	6.78	1	9.07	0.75
8	2	7.42	0	9.07	0.82
9	3	8.32	0	9.07	0.92
10	0	8.32	0	9.07	0.92

같은 예시의 완성 표이다. 4~6위의 0 때문에 점수가 낮아졌다가, 이후 순위에서 정답을 회수하며 0.92까지 회복한다.

정규화 덕분에 정답 수가 다른 문항들도 동일한 기준으로 측정할 수 있고, 평가셋 전체의 평균도 낼 수 있다.



NDCG 해석의 세 가지 규칙

정의보다 중요한 것은 읽는 법이다. NDCG는 세 가지 규칙과 함께 사용할 때 의미가 생긴다.

1 K는 컨텍스트 크기에 맞춘다

컨텍스트에 넣는 청크 수가 3이면 NDCG@3을 본다.

K가 다른 점수끼리는 비교할 수 없다. 평가셋 전체에서 K를 고정한 뒤 비교한다.

청크 3개 투입



NDCG@3

2 등급 라벨이 있을 때 제대로 드러난다

이진 평가셋에서도 계산은 되지만, 순서 품질은 등급 관련도일 때 제대로 드러난다.

등급은 사람 라벨링이나 LLM Judge 채점으로 얻는다. 사람이 라벨을 부여할 때는 라벨러 간 일치도를 함께 확인한다.

3

2

1

0

등급 라벨

3 Recall과 짝으로 읽는다

Recall이 같은데 NDCG가 올랐다면, 검색된 문서는 같고 배치가 좋아진 것이다.

재순위(rerank)·하이브리드 검색처럼 순서를 바꾸는 개선의 효과를 측정하는 표준 지표이다.

Recall =

NDCG ↑

한 지표로 요약하지 않는다. Recall은 "검색했는가", NDCG는 "잘 배치했는가"를 측정한다.

정답 수와 순위의 중요도, 관련도 등급 유무에 따라 주 지표가 달라진다.

상황	정답과 관련도	주 지표	함께 볼 지표
정답이 하나, 몇 위에 오는지가 중요하다	정답 1개 · 이진	MRR	Hit Rate@K
정답이 여러 개, 빠짐없이 회수하는 것이 중요하다	정답 여러 개 · 이진	Recall@K	Precision@K
정답이 여러 개, 상위 배치까지 중요하다	정답 여러 개 · 이진	MAP	Recall@K
근거의 유용성에 등급이 있다	등급 (0·1·2)	NDCG@K	Recall@K

지표를 여러 개 늘리기보다 주 지표 하나와 보조 지표 하나를 정해 두면 판단이 빨라진다.

평가셋 점수는 문항별 점수의 평균이다. 평균이 같아도 실패의 성격은 문항마다 다르다.

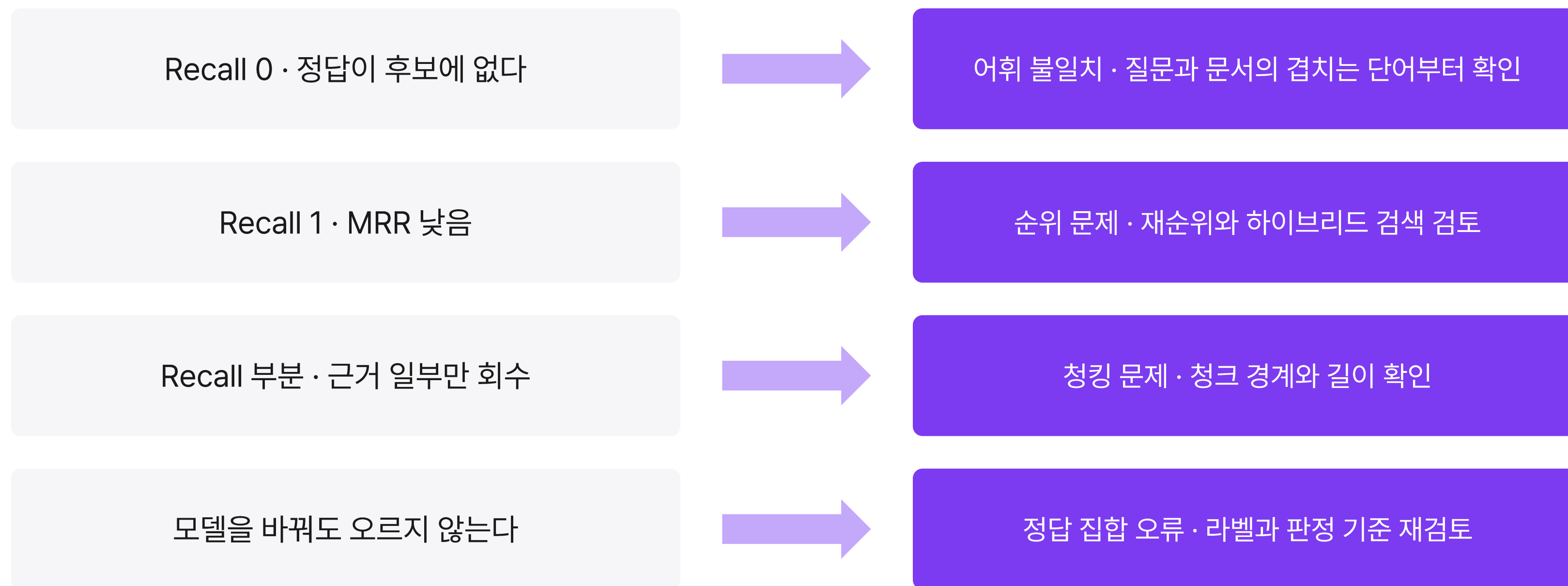
문항	Recall@3	RR	판정	다음 확인
재구매 고객 LTV 배수	1.00	1.00	정상	-
라이브커머스 1위 서비스	1.00	0.33	순위 밀림	재순위·질의 재작성
중국 라이브커머스 규모·전망	0.50	0.50	근거 일부 누락	청킹·K 확대
럭셔리 시장 성장률	0.00	0.00	검색 실패	인덱스 누락·텍스트 추출

평균 Recall@3은 0.63이지만 네 문항의 실패 성격은 모두 다르다. 평균 한 숫자만으로는 다음 조치를 정하기 어렵다.

읽는 순서: RR이 0인 문항 → Recall이 1보다 작은 문항 → RR이 낮은 문항.



지표 조합을 보면 수정할 지점의 범위가 좁혀진다.



지표로는 원인을 특정할 수 없다. 패턴으로 후보를 좁힌 뒤 문항 원문을 열어 확인한다.

청킹 설정과 청크 품질

기법마다 분할 기준과 속도·비용이 다르다. 문서와 평가 결과에 맞춰 고른다.

기법	청킹 기준	특징과 한계
Character	고정 글자 수	단순하고 빠르지만 문장 중간 절단이 잦다
Recursive	문단→문장→단어 재귀	문서 구조를 최대한 유지하는 기본값이며, 구분자가 없는 문서에서는 효과가 떨어진다
Semantic	임베딩 유사도 경계	의미 단위로 나뉘지만 임베딩 호출이 필요해 느리다
Agentic	LLM이 분할을 판단	복잡한 문서에 유연하지만 처리 속도가 가장 느리고 비용이 크다

권장: 특별한 이유가 없다면 Recursive에서 출발해 평가로 조정한다. 실습은 앞 두 기법을 설정별로 비교한다.



의미 기반 청킹의 동작 원리

인접 문장의 임베딩 유사도가 떨어지는 지점을 경계로 삼는 방식이다.

동작 순서

1. 문서를 문장 단위로 나눈다
2. 문장마다 임베딩을 계산한다
3. 인접 문장 쌍의 유사도가 크게 떨어지는 지점에서 분할한다

임계값과 적용 조건

- 경계 판정은 유사도 하락 폭의 백분위로 정하는 방식이 흔하다
- 문장 수만큼 임베딩 호출이 필요해 더 느리고 비용이 든다
- 주제가 여러 개 섞인 긴 문서에서 효과가 크다

```
from langchain_experimental.text_splitter import SemanticChunker
splitter = SemanticChunker(embeddings, # 임베딩 모델을 주입한다
                           breakpoint_threshold_type="percentile", breakpoint_threshold_amount=95)
```

임베딩 호출 비용이 추가되므로, 고정 길이 분할과 검색 지표를 비교한 뒤 채택하는 것을 권장한다.

청크 크기·overlap의 trade-off

크기와 overlap을 조정하면 응집도·Recall·토큰 수가 서로 다른 방향으로 움직인다.

설정 변경	응집도	검색 Recall	컨텍스트 토큰	주의
청크를 짧게	올라간다	근거가 잘리면 내려간다	같은 K에서 줄어든다	문맥이 끊긴 청크가 늘어난다
청크를 길게	내려간다	한 청크에 근거가 모두 담겨 올라갈 수 있다	늘어난다	무관한 내용이 함께 들어온다
overlap을 늘림	경계 손실이 줄어든다	경계에 걸친 근거를 살린다	중복 저장으로 늘어난다	중복 청크 탐지로 검출된다
overlap을 줄임	변화가 작다	경계에 걸친 근거를 놓칠 수 있다	줄어든다	문장 절단이 드러난다

K × 청크 길이가 컨텍스트 토큰을 정한다. 청크를 길게 하면 같은 K에서도 입력 토큰이 늘어 비용과 지연이 함께 오른다.



청킹 품질 지표 세 가지



좋은 청킹의 조건: 청크 안은 한 주제로 묶이고, 청크 사이는 주제가 나뉘고, 문장은 잘리지 않는다.

원문 문서



↓ 청킹

청크 A · 라이브커머스 정의

- 라이브커머스는 방송과 전자상거래의 결합이다.
- 시청 중 실시간으로 상품을 주문하는 방식이다.
- 거래액은 2023년 10조 원에 이를 전망이다.

청크 B · 이용자 특성

- 이용 경험률은 30대가 66.2%로 가장 높다.
- 주로 구매하는 상품은 식품이 54.8%이다.
- 가장 많이 쓰는 플랫폼은 네이버 쇼핑라이브이다.



① 내부 응집도 ↑

한 청크 안 문장들의 평균 유사도

② 경계 유사도 ↓

인접 청크 사이의 유사도, 낮을수록
경계가 주제 전환점에 놓인 것

잘못된 청킹의 예

"...2023년 10조 원에 이" (문장 절단)

③ 문장 절단을 → 0

세 지표 모두 임베딩·문자열 계산만으로 산출된다. LLM 없이 청킹 설정을 비교할 수 있다.

같은 문서·같은 임베딩 모델로 측정해야 설정 간 비교가 공정하다. 임베딩 모델이 다르면 응집도와 경계 유사도의 값 자체가 달라진다.

실습에서 세 지표를 직접 구현하고, 가중치를 바꾸면 순위가 바뀌는 것까지 확인한다.



품질 지표 구현 코드

실습 · 품질 지표

```
def cohesion(chunk):          # 내부 응집도
    sents = split_sentences(chunk)
    if len(sents) < 2: return 1.0
    sim = cosine_similarity(embed(sents))
    return sim[np.triu_indices_from(sim, k=1)].mean()

def boundary_sim(chunks):     # 경계 유사도
    if len(chunks) < 2:
        return float("nan")   # 0.0으로 두면 안 자른 설정이 이긴다
    e = embed(chunks)
    return np.mean([cosine_similarity(
        e[i:i+1], e[i+1:i+2])[0, 0]
        for i in range(len(e) - 1)])

def ends_sentence(chunk):     # . ! ? 또는 보고서 문체의 ~함/~임/~됨
    return bool(re.search(r"[.!? 。임함됨]$", chunk.strip()))

def cut_rate(chunks):        # 문장 절단율
    cut = sum(1 for c in chunks if not ends_sentence(c))
    return cut / len(chunks)
```

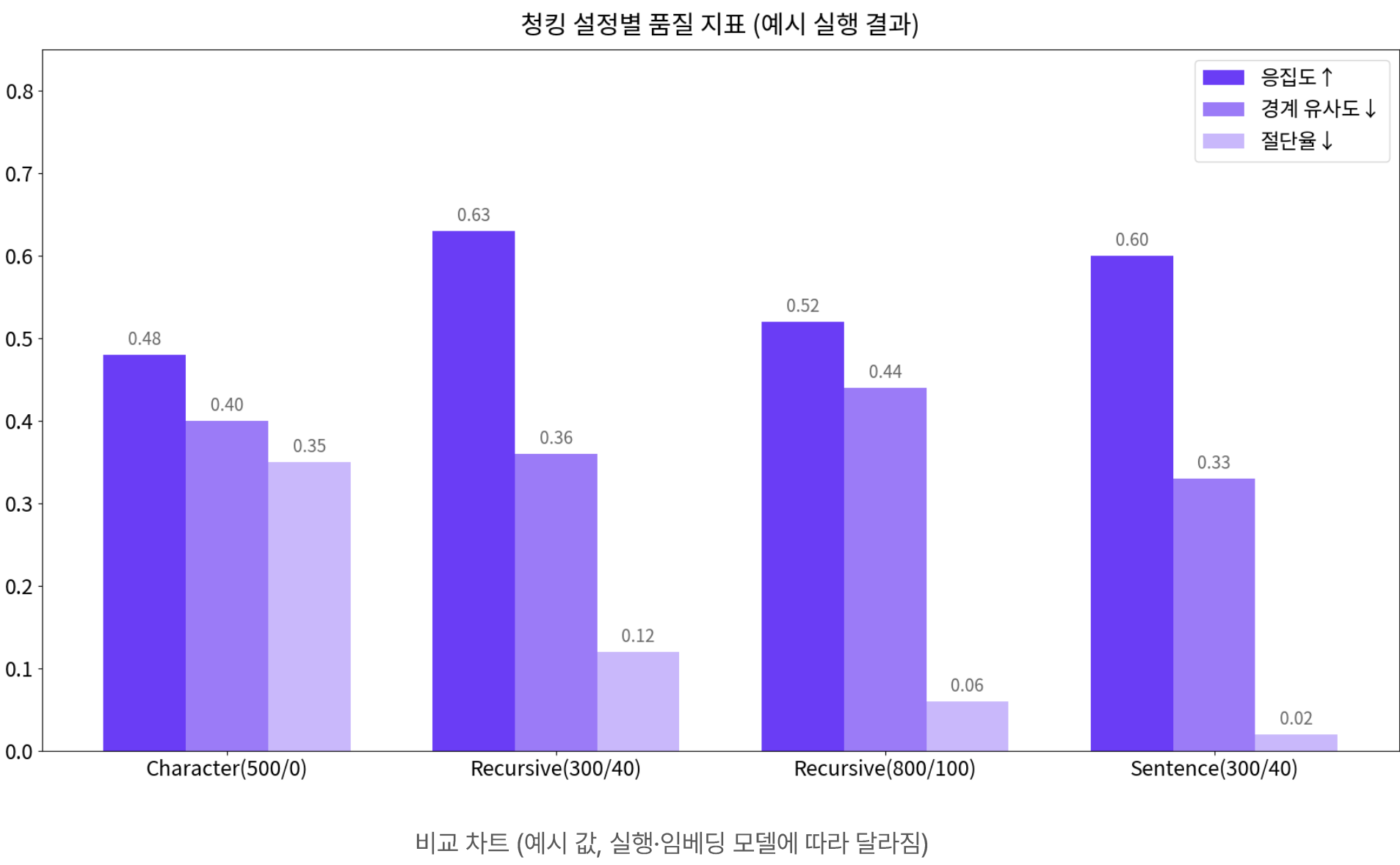
- 응집도는 청크 내부 문장 쌍의 평균 유사도이다. 문장이 하나면 1.0으로 둔다.
- 경계 유사도는 청크가 하나뿐이면 nan을 반환한다. 0.0으로 두면 분할하지 않은 설정이 유리해진다.
- 절단율은 임베딩 없이 문자열 검사만으로 계산되는 비용이 가장 낮은 지표이다. 실습 코퍼스의 보고서 문체는 "~함"으로 끝나는 문장이 많아 문장 끝 규칙에 포함한다.

같은 임베딩 모델로 같은 문서를 측정해야 기법 간 비교가 공정해진다.



기법별 측정 결과

숫자보다 방향(↑ ↓)과 대소 관계를 본다. 네 설정을 같은 문서로 측정해 나란히 놓는다.



Character(500/0)

절단율이 높다
공백 기준 분할이 문장을 자른다

Sentence(300/40)

절단율이 가장 낮다
문장 끝에서 자르고, overlap은 경계 유사도를 올린다

측정 지표
응집도↑ · 경계 유사도↓ · 절단율↓

한 지표에서 높은 설정이 다른 지표에서는 낮게 나올 수 있다. 종합 순위는 가중치에 따라 달라진다.



차원 축소 기법: t-SNE vs UMAP

실습에서는 같은 임베딩을 두 기법으로 투영해, 두 결과에서 함께 묶이는 조합만 근거로 사용한다.

t-SNE

- 가까운 이웃 관계(국소 구조)를 잘 보존한다
- 군집 내부의 세밀한 구조를 잘 드러낸다
- 실습 4에서 UMAP과 나란히 비교한다. 느리고, 군집 사이 거리는 해석하면 안 된다

UMAP

- 국소 구조와 함께 전역 구조도 비교적 잘 보존한다
- 대규모 데이터에서 빠르고, n_neighbors·min_dist로 조절한다
- 실습 기본 투영. 단, 두 기법 모두 축 자체에는 의미가 없다

```
from sklearn.manifold import TSNE          # perplexity는 표본 수보다 작아야 한다 (기본값 30은 작은 데이터에서 실패)
tsne = TSNE(n_components=2, perplexity=min(5, len(chunks) - 1),
            metric="cosine", init="random", random_state=42)  # 고차원 임베딩에는 init="random"이 안전
```

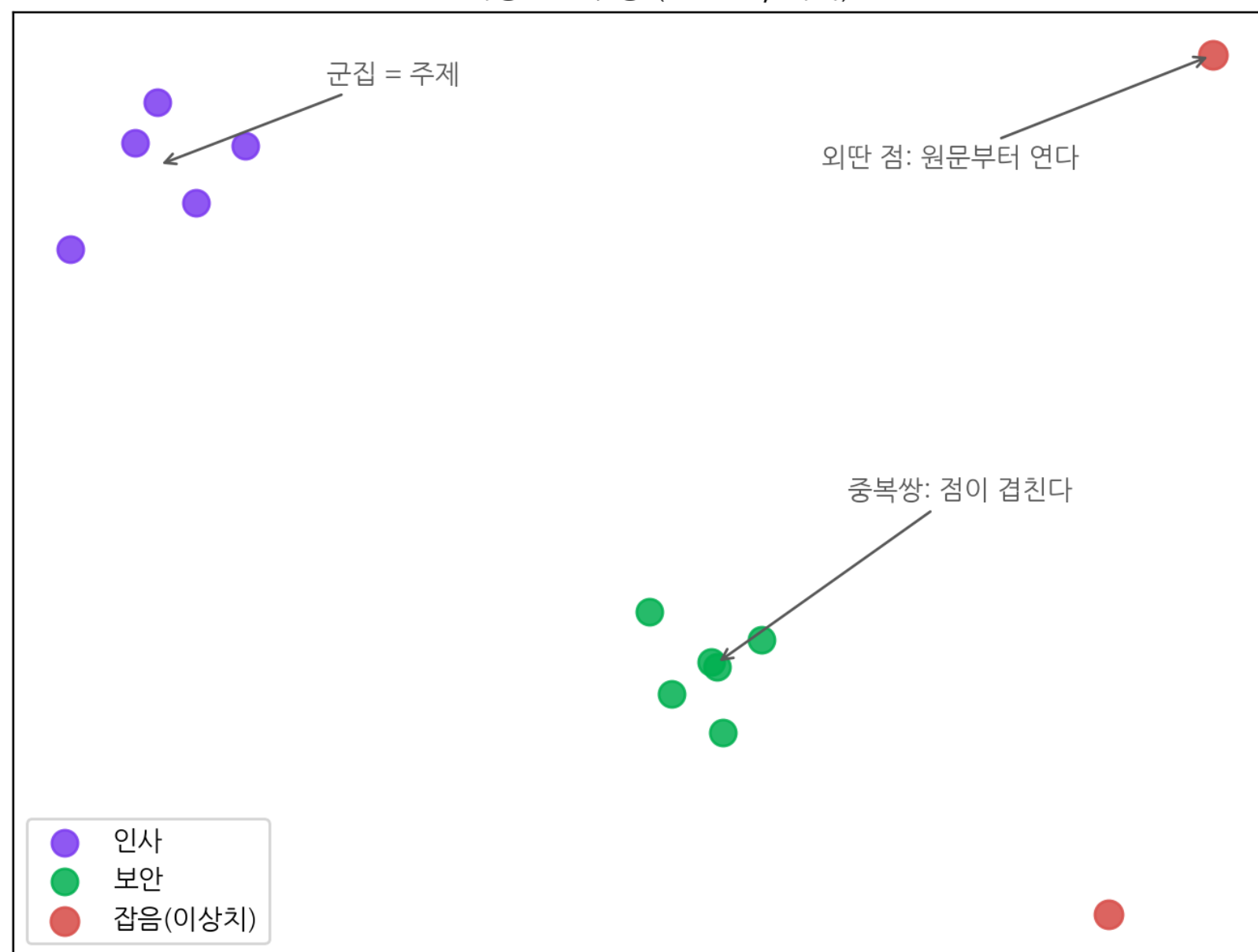
점 사이 거리·군집 배치는 투영의 부산물이므로, 두 그림에서 같은 것만 근거로 사용한다.



UMAP 산점도 읽는 법

임베딩을 2차원으로 투영하면 군집·중복·이상치가 한 그림에 드러난다.

임베딩 2D 투영 (UMAP, 예시)



실습 4에서 생성하는 산점도 (예시)

군집

뭉친 점들은 같은 주제의 군집이다

주제별로 색을 칠해 군집이 섞이지 않는지 확인한다

중복쌍

거의 겹친 두 점은 중복쌍이다

유사도 임계값 탐지와 교차 확인한다

이상치

외딴 점을 가장 먼저 확인한다

깨진 인코딩·조각 청크가 주로 발견된다



네 설정을 한 UMAP 공간에 투영

네 설정의 청크를 같은 UMAP 공간에 투영해야 비교가 성립한다.
설정별로 따로 투영하면 좌표의 의미가 달라진다.

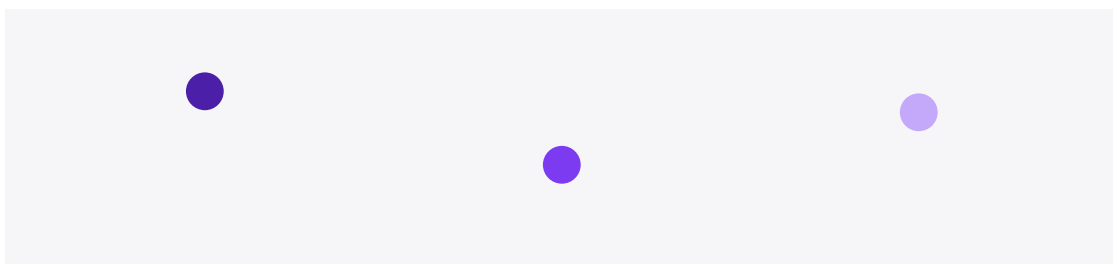
같은 UMAP 공간 · fit_transform은 한 번만, 설정별 패널은 축을 공유한다 · 점의 색은 출처 쪽

● 1쪽

● 2쪽

● 3쪽 ...

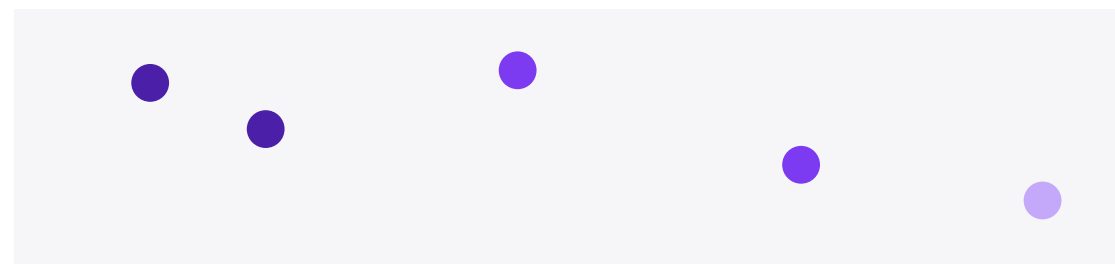
패턴 1 · 쪽별로 나뉜다



1 쪽 단위로 점이 모인다

같은 쪽의 청크가 가까이 놓여 경계가 뚜렷하다

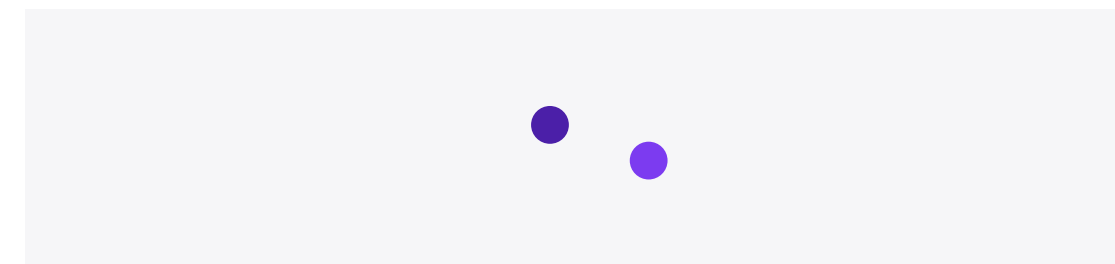
패턴 2 · 경계를 넘는 점



2 두 쪽 사이에 놓인 점이 있다

쪽 경계를 넘어 잘린 청크의 절단 위치를 다시 본다

패턴 3 · 한 점에 뭉친다



3 청크가 크면 점이 몇 개뿐이다

여러 쪽의 내용이 한 청크에 섞여 쪽 구분이 드러나지 않는다

```
all_chunks = [c for chunks in techniques.values() for c in chunks] # 네 설정을 모아 한 번만
reducer = umap.UMAP(n_neighbors=min(8, len(all_chunks) - 1), min_dist=0.15,
                    metric="cosine", random_state=42)
xy = reducer.fit_transform(embed(all_chunks))
fig, axes = plt.subplots(1, 4, sharex=True, sharey=True) # 축을 공유해 설정별 패널
```

점의 색(출처 쪽)은 키워드가 아니라 문서 내 위치(pages_of)로 정한다. 쪽 머리글이 반복되기 때문이다.
분포는 지표를 해석하는 도구이다. 지표만으로는 "왜 그 값인가"에 답할 수 없다.



가중치별 순위 변화

가중치 조합	응집도 w	경계 w	절단율 w	1위 (예시)
절단율 중시	0.5	-0.3	-2.0	Sentence(300/40)
응집도 중시	2.0	-0.5	-0.3	Recursive(300/40)
경계 중시	0.5	-2.0	-0.5	Sentence(300/40)
균형	1.0	-0.7	-1.0	Sentence(300/40)

가중치는 팀의 우선순위이다. 조합마다 상위 설정이 달라지면 설정 간 지표 차이가 크지 않다는 신호이다.



청킹 지표의 한계와 최종 판정

응집도와 경계 유사도는 대리 지표이다. 목적은 검색 품질을 높이는 것이다.

대리 지표 (청킹 품질 지표)

- 청크 내부가 얼마나 한 주제인지를 본다
- 임베딩만 있으면 평가셋 없이 계산된다
- 설정 후보를 좁히는 데 쓴다

평가셋 없이 계산

목적 지표 (검색 지표)

- 질문에 필요한 근거가 검색되는지를 본다
- 정답 근거가 라벨된 평가셋이 필요하다
- 최종 채택의 근거가 된다

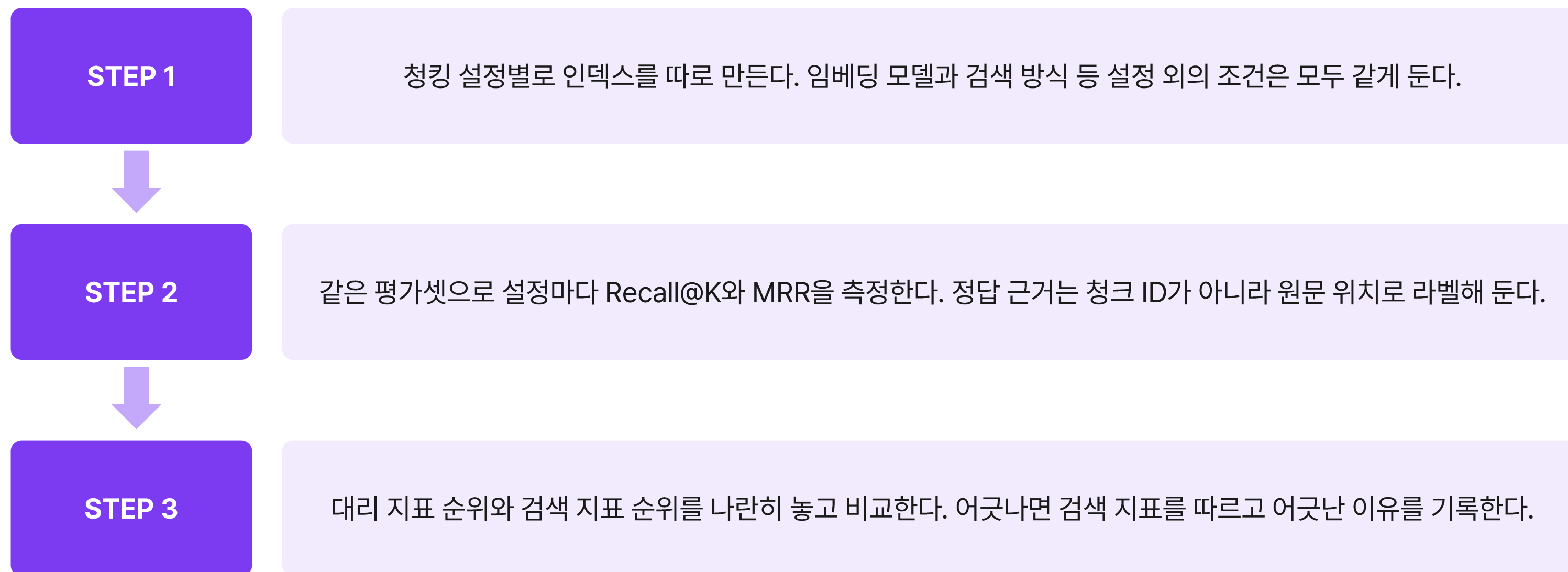
평가셋 필요

대리 지표가 좋아도 검색 지표가 낮으면 채택하지 않는다. 두 순위가 어긋나면 검색 지표를 따른다.



청킹 설정과 검색 지표의 연결 검증

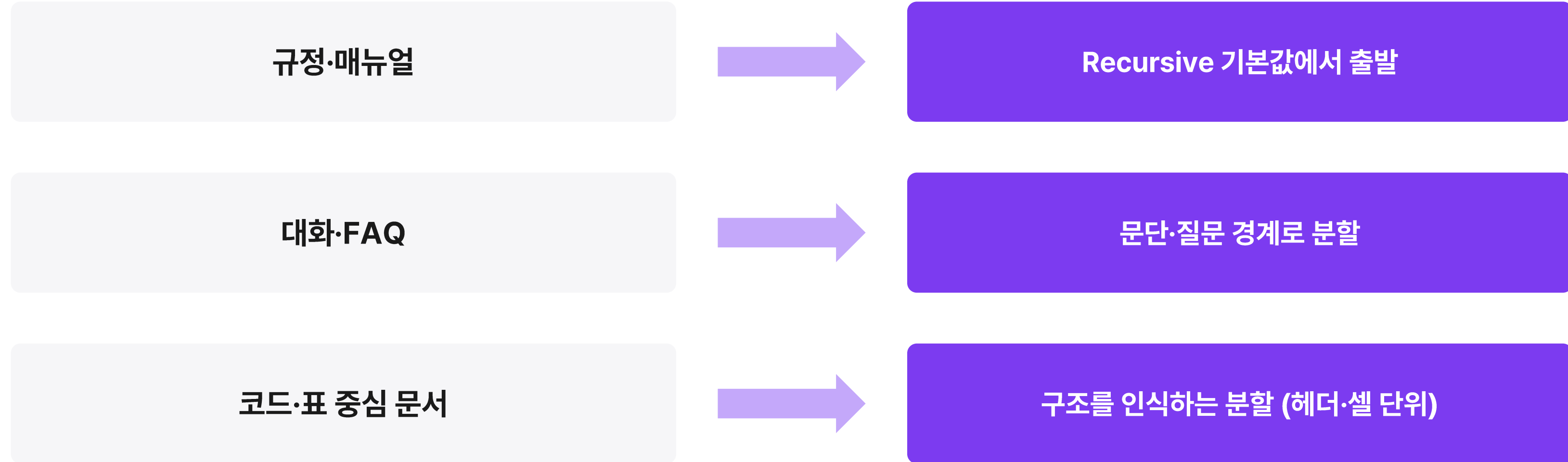
설정을 바꿀 때마다 검색 지표를 다시 측정하는 절차를 정해 둔다.



정답 근거를 원문 위치로 라벨해 두면 청킹 설정을 바꿔도 평가셋을 다시 만들지 않는다.



문서 구조에 따라 출발 전략이 달라진다.
어느 전략을 채택할지는 같은 평가셋의 검색 지표로 판정한다.





표·목록·코드가 있는 문서의 청킹

TYPE 01

표

- 행 단위로 잘리면 열 이름이 사라져 숫자만 남는다. 헤더를 붙여 셀 묶음 단위로 한 청크로 만든다.

TYPE 02

목록·번호 항목

- 항목 중간에서 잘리면 조건의 일부만 남는다. 항목 경계에서 분할하고 상위 제목을 청크 앞에 붙인다.

TYPE 03

코드 블록

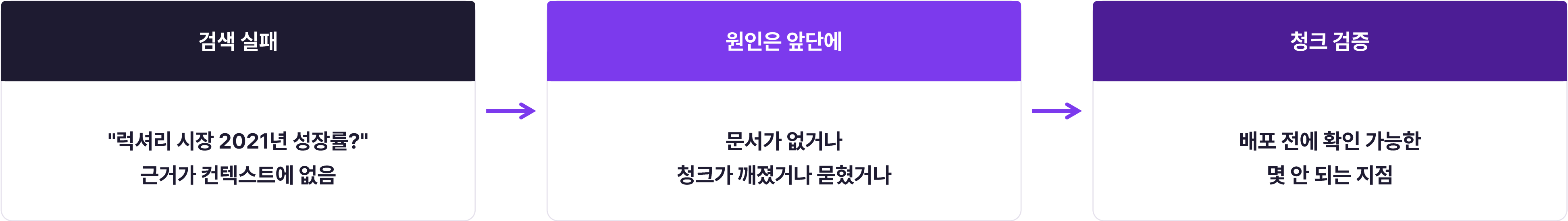
- 블록 중간에서 잘리면 실행되지 않는 조각이 남는다. 블록 단위로 분할하고 언어 표시와 함께 둔다.

구조 손상은 길이 분포와 절단율에서 먼저 드러난다.
경계를 인식하는 분할기를 먼저 적용한 뒤 길이 기준을 적용한다.



청크 검증의 위치

검색 실패로 분류된 질문이 청크 검증의 출발점이다.



청크 검증의 진단 순서 (각 단계의 지표는 단계별로 따로 정의한다)

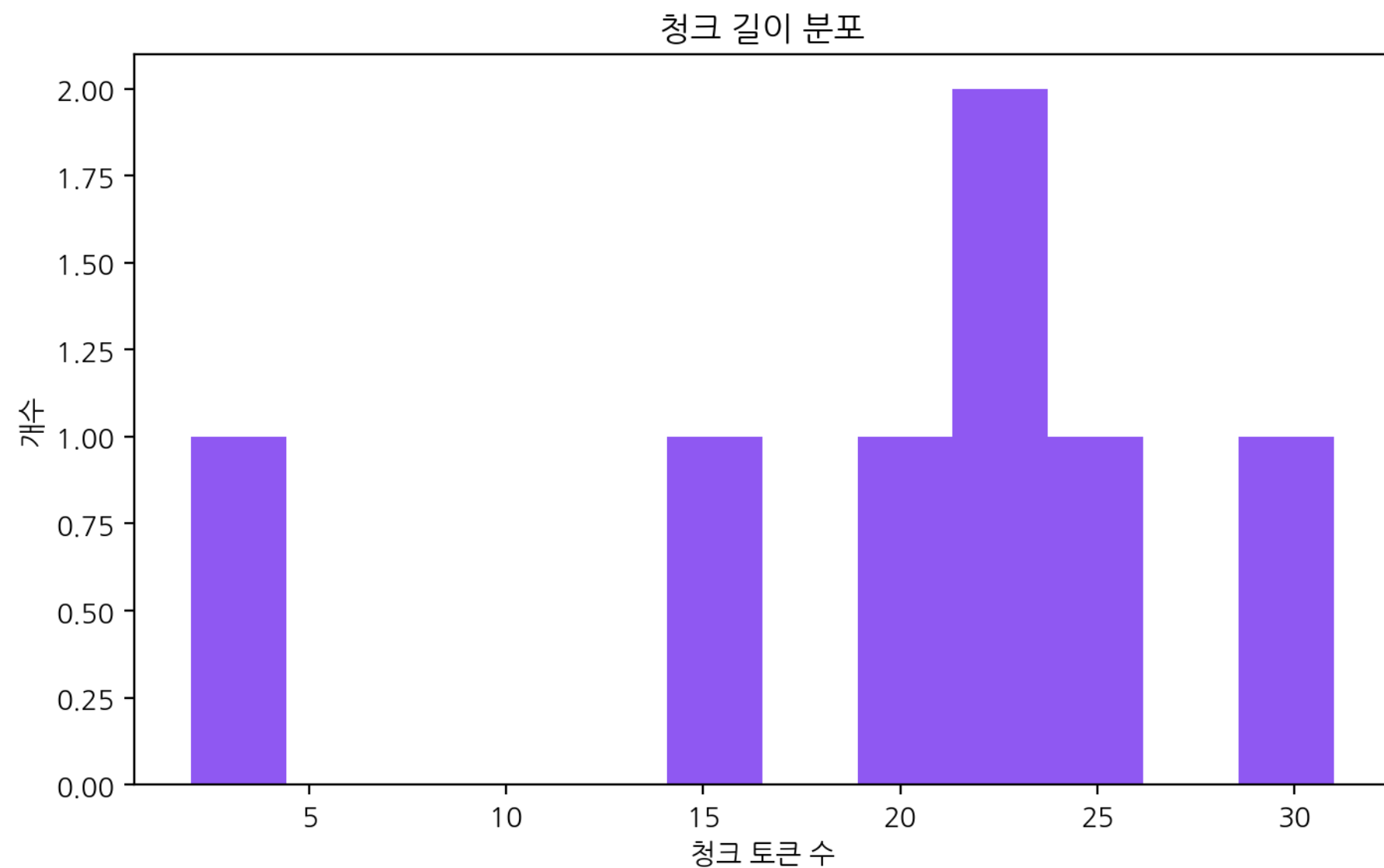


일곱 단계 모두 LLM 호출 없이 계산만으로 확인한다. 비용 대비 효과가 큰 검증이다.



청크 길이 분포 진단

색인된 청크의 길이 분포에서 평균이 아니라 양쪽 꼬리를 본다.



실습에서 생성하는 히스토그램 (수치는 실행마다 다름)

청크 길이를 토큰 수로 측정해 히스토그램으로 그리면 청크 집합의 상태를 한눈에 파악할 수 있다. 왼쪽 꼬리에 놓인 10토큰 미만의 조각 청크(머리글·페이지 번호)는 검색 노이즈가 된다. 오른쪽 꼬리에 놓인 지나치게 긴 청크는 분할 실패이며, 여러 주제가 섞여 있다. 평균·중앙값에는 양극단이 드러나지 않는다. 분포로 보고 극단값의 원문을 확인한다.

각주 · 길이는 토큰 수로 측정한다

```
enc = tiktoken.get_encoding("o200k_base")
```

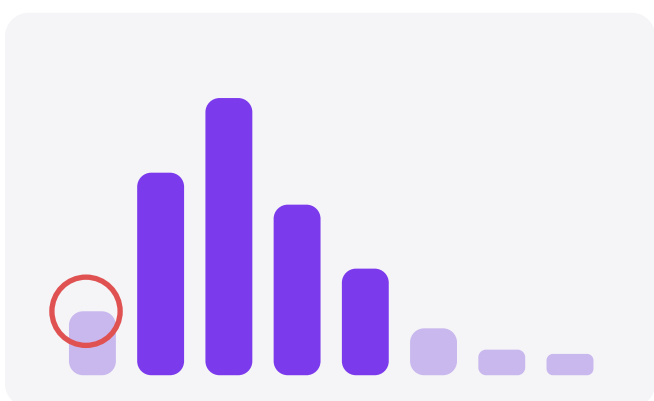
문자 수가 아니라 이 인코딩의 토큰 수를 센다. 비용·컨텍스트 한도와 단위가 맞다.



청크 진단 파이프라인

배포 전에 청크 집합을 일곱 단계로 진단한다. 모두 LLM 호출 없이 계산만으로 수행한다.

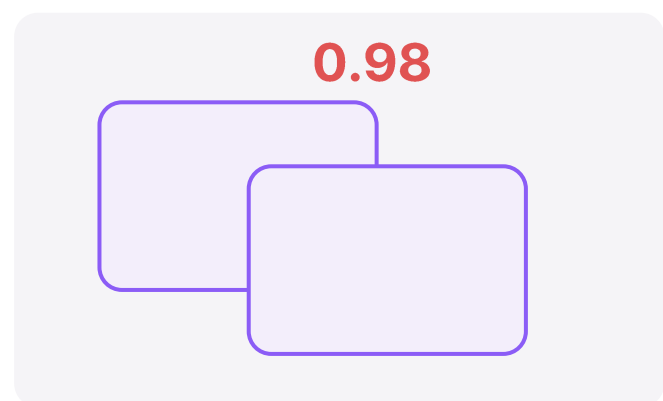
1 길이 분포



히스토그램의 양쪽 꼬리

10토큰 미만 조각
잘리지 않은 덩어리

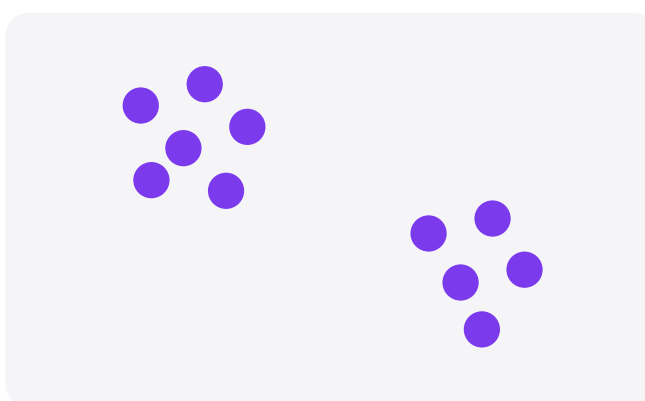
2 중복 탐지



코사인 유사도 > 0.95

같은 내용이 Top-K 자리를 차지

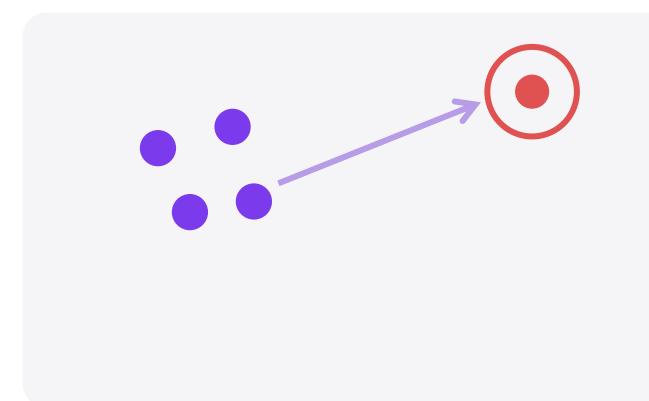
3·4 분포 시각화



UMAP · t-SNE 투영

군집 = 주제
두 투영의 조합 일치 확인

5·6 이상치·군집 검증



중심 거리 · 군집 지표

깨진 인코딩·조각 머리글
silhouette·ARI로 검증

7 정제 규칙



is_valid() 필터

재발 방지, 인덱싱 전에 상시 적용

왜 이 순서인가

비용이 낮은 진단부터 큰 진단으로 진행하는 순서이다. 투영 두 가지로 교차 확인하고 군집 지표로 검증한 뒤 규칙으로 정리한다.

실습에서 구현

진단으로 끝내지 않고 마지막 단계에서 규칙으로 만드는 것이 핵심이다.
같은 문제가 다시 발생하는 것을 막을 수 있다.



청크 메타데이터 검증

본문만 검증해서는 부족하다. 청크에 붙는 메타데이터도 검증 대상이다.

source

원본 문서를 가리킨다. 답변에 출처를 붙이는 기준이며, 없으면 인용 표시를 만들 수 없다.

page

쪽 번호이다. 사용자가 원문을 찾아갈 수 있게 하며, 없으면 문서 전체를 다시 읽어야 한다.

section

제목 경로이다. 메타데이터 필터와 라우팅의 조건이며, 없으면 범위를 좁힌 검색을 실행할 수 없다.

chunk_id

청크 식별자이다. 평가셋의 정답 근거가 가리키는 값이며, 없으면 검색 지표를 계산할 수 없다.

doc_version

문서 버전이다. 갱신 전후를 구분하며, 없으면 낡은 청크와 최신 청크가 섞인다.

indexed_at

색인 시각이다. 오래 갱신되지 않은 청크를 찾아내며, 없으면 인덱스 노후를 감지하기 어렵다.



중복 청크 탐지

중복 탐지

```
from sklearn.metrics.pairwise import cosine_similarity

sim = cosine_similarity(emb)
np.fill_diagonal(sim, 0)

dups = []
for i in range(len(chunks)):
    for j in range(i + 1, len(chunks)):
        if sim[i, j] > 0.95:
            dups.append((i, j, float(sim[i, j])))
```

- 임베딩 코사인 유사도 행렬에서 대각선(자기 자신)을 지우고 모든 쌍을 확인한다.
- 1.0은 완전 중복, 0.95 초과는 준중복 후보이다. 임계값을 낮추면 오탐이 늘어난다.
- 전수 비교는 $O(n^2)$ 이라 규모가 크면 ANN으로 대체한다.

중복은 저장 공간을 낭비할 뿐 아니라,
Top-K 자리를 같은 내용으로 채워 검색 다양성을 해치는 원인이 된다.



중복 판정 임계값 정하기

임계값을 먼저 정하지 않는다.

쌍별 유사도 분포를 그려 값이 몰린 구간을 확인한 뒤, 구간마다 처리 방법을 다르게 정한다.

0.99 이상 · 거의 동일



자동 제거 후보
원문 확인 없이 정리해도 위험이 낮다

0.95 ~ 0.99 · 준중복



원문을 열어 판단
머리글 반복과 실제 중복이 섞여 있다

0.90 ~ 0.95 · 주제 유사



제거하지 않는 것을 권장
제거하면 Recall이 떨어질 수 있다



이상치 탐지 세 단계

STEP 1

전체 임베딩의 중심(centroid)을 구하고, 각 청크의 중심 거리(norm)를 계산한다.

STEP 2

거리 상위 청크를 추려 목록으로 만든다. 실습에서는 상위 3개를 본다.

STEP 3

원문을 열어 원인을 확인한다. 깨진 인코딩, 조각 머리글, 표·목차 잔재가 대부분이다.



군집화 검증: 산점도를 숫자로

산점도는 눈으로 보는 도구이다. 같은 구조를 숫자로 확인하려면 군집화를 사용한다.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score

K = 3 # 주제 수를 알고 있으므로 3
km = KMeans(n_clusters=K, n_init=10, random_state=42).fit(emb)

sil = silhouette_score(emb, km.labels_, metric="cosine")
ari = adjusted_rand_score(labels, km.labels_)
for c in range(K): # 군집별 주제 구성
    idx = [i for i, l in enumerate(km.labels_) if l == c]
    print(len(idx), Counter(labels[i] for i in idx))
```

해석 1 ARI가 낮다

임베딩이 주제를 구분하지 못하는 것이므로 모델 교체를 검토한다.

silhouette

-1 ~ 1

군집 분리도

군집이 서로 얼마나 잘 분리됐는지를 본다. 1에 가까울수록 경계가 뚜렷하다. 코사인 거리 기준으로 계산한다.

ARI

-0.5 ~ 1

주제 라벨과의 일치도

군집이 실제 주제 라벨과 얼마나 일치하는지를 본다. 1이면 완전 일치, 0 근처면 무작위 배정 수준이다. 라벨이 있어야 계산된다.

해석 2 silhouette만 낮고 ARI는 높다

주제는 맞게 나뉘었으나 경계가 흐린 것이므로 경계 청크의 문장을 다시 자른다 (실습 3의 경계 유사도와 연결).

군집화는 산점도에서 본 구조를 숫자로 바꾸는 도구이다. 다만 숫자 하나로 품질을 판정하지는 않는다.



두 값은 범위와 전제가 다르다. 낮게 나오는 것이 정상인 경우도 있다.

silhouette		ARI
판단 기준 0.5 이상이면 경계가 뚜렷한 편, 0.25 이하면 구조를 읽기 어렵다		판단 기준 0 근처면 무작위 배정 수준, 1에 가까우면 라벨과 일치한다
전제 주제 라벨 없이 군집 결과만으로 계산된다	구분	전제 주제 라벨이 있어야 계산할 수 있다
낮게 나오는 정상 사례 문서가 한 주제로만 구성된 경우		낮게 나오는 정상 사례 라벨 기준과 군집 수가 어긋난 경우

두 값이 낮아도 청킹 실패는 아니다. 원문을 열어 군집 구성이 납득되는지 함께 확인한다.

이상치를 찾은 다음에는 규칙으로 바꾼다. 같은 증상이 반복되기 때문이다.

증상	정제 전 예시	정제 규칙과 결과
인코딩 오류	iŠǎí,ǎ처럼 깨진 문자	추출 단계에서 인코딩을 지정해 다시 추출한다. 깨진 문자 비율이 높은 청크는 색인에서 제외한다
머리글·바닥글 반복	모든 쪽 첫머리에 "중국 콘텐츠 산업동향 (2021년 18호)"	쪽마다 반복되는 줄을 추출 단계에서 제거한다. 반복 문구가 유사도를 인위적으로 올리는 원인이 사라진다
표 잔재	구분선과 셀 기호만 남은 조각	표는 헤더를 붙여 셀 묶음 단위로 한 청크로 만들거나, 문장 형태로 변환한 뒤 분할한다
목차 조각	제목과 쪽 번호만 나열된 청크	목차·색인 페이지는 색인 대상에서 제외한다. 길이가 짧고 숫자 비율이 높은 청크로 걸러낸다

정제 규칙은 파이프라인에 넣어 자동으로 적용한다. 매번 직접 수정하는 방식으로는 다음 문서에서 같은 문제가 다시 나온다.

임베딩 모델과 색인 설정 비교



색인 설정이 Recall에 미치는 영향

같은 임베딩 모델과 같은 K에서도 색인 설정에 따라 측정되는 Recall이 달라진다.

M 연결 수 (M)	구 구축 폭 (ef_construction)	검 검색 폭 (ef_search)
노드마다 유지하는 이웃 수이다. 크게 하면 Recall이 오르고 메모리 사용량도 함께 늘어난다	색인을 만들 때 탐색하는 후보 폭이다. 크게 하면 색인 품질이 오르고 구축 시간이 늘어난다	질의마다 탐색하는 후보 폭이다. 크게 하면 Recall이 오르고 질의 지연이 늘어난다

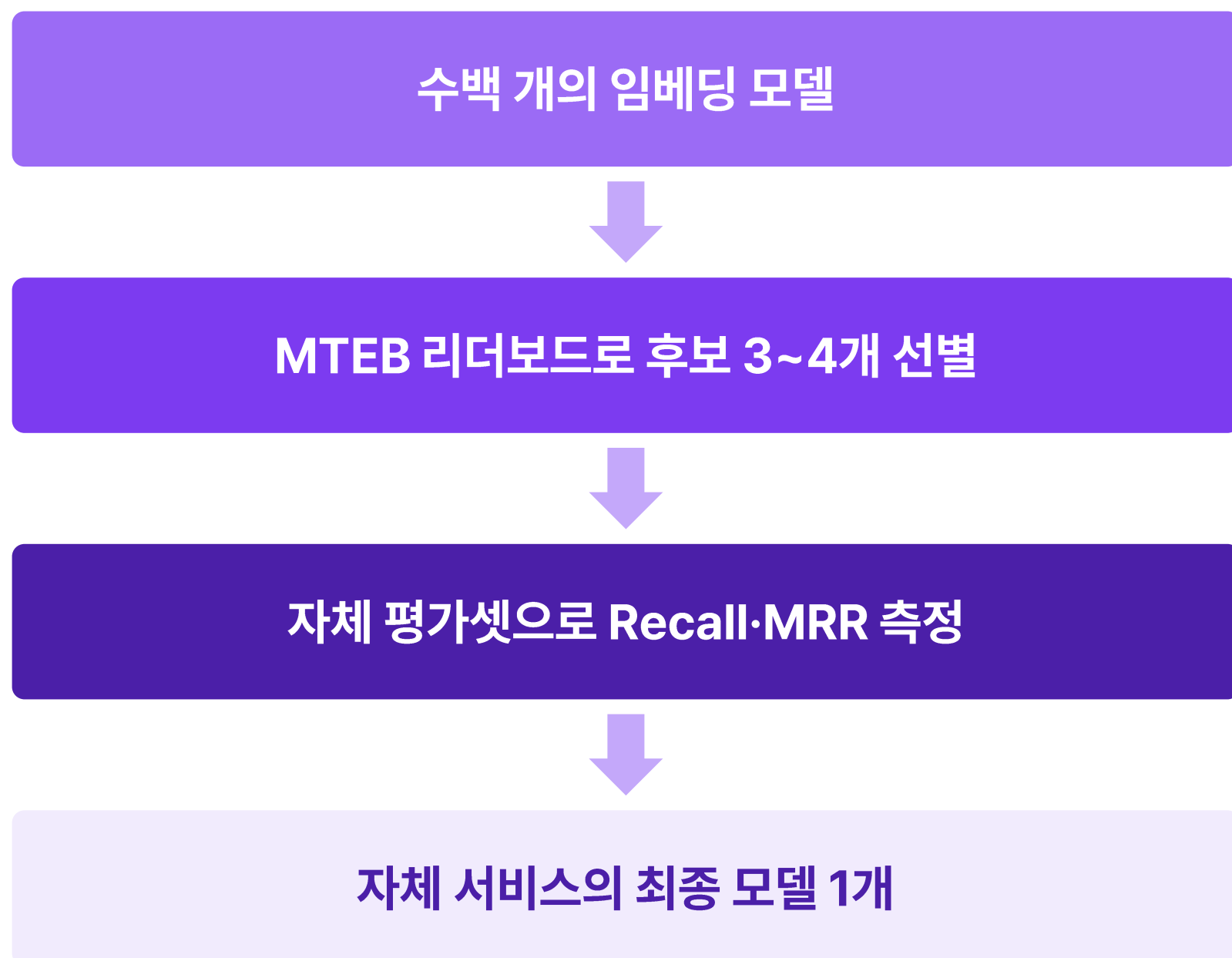
측정할 때의 유의 사항

- 색인 설정을 바꾸면 같은 평가셋에서도 Recall이 달라진다. 모델을 비교할 때는 설정을 고정한다
- 정확 탐색으로 측정한 값이 그 색인의 상한이며, 근사 탐색 설정마다 상한에 얼마나 가까운지가 달라진다
- 문서가 수십 건인 실습 규모에서는 차이가 드러나지 않을 수 있다. 규모가 커질 때 다시 확인한다

실습에서 쓰는 Chroma의 기본 거리 함수는 제곱 L2이며, 코사인을 쓰려면 컬렉션을 만들 때 지정해야 한다.



MTEB 벤치마크



- MTEB는 검색·분류·문장 유사도 등 여러 태스크로 임베딩 모델을 비교하는 공개 벤치마크이다. 리더보드에서 수백 개 모델의 성능을 한눈에 확인한다.
- MTEB의 역할은 후보를 몇 개로 좁히는 필터에 그친다. 리더보드의 코퍼스와 자체 문서·질문은 분포가 다르기 때문이다.



임베딩 모델 선택 절차

벤치마크는 후보를 좁히는 도구이고, 채택은 자체 평가셋의 수치로 판단하는 것을 권장한다.

① 후보 축소

공개 벤치마크에서 언어와 태스크가 맞는 모델 3~4개로 줄인다.
리더보드 순위는 여기까지만 쓴다.

② 자체 평가셋 측정

같은 평가셋·같은 K·같은 색인 설정으로 Recall@K·MRR·NDCG
를 측정하고, 문항별 결과까지 확인한다.

③ 운영 조건 확인

차원과 저장 비용, 최대 입력 길이, 자체 호스팅 가능 여부와 과금
방식을 함께 본다.

④ 채택과 재인덱싱

교체하면 전체 문서를 다시 인덱싱해야 한다. 일회성 비용과 적용
시점을 함께 계획한다.

네 단계에서 확인하는 것이 각각 다르다. 어느 하나를 건너뛰면 교체한 뒤에 되돌리기 어려운 문제가 남는다.



임베딩 모델 후보군 비교

계열	제공 형태	다국어	특징	고려할 점
OpenAI text-embedding-3	API	지원	범용 성능과 운영 편의성	호출량 과금, 외부 전송
Cohere Embed	API	다국어 특화	검색 용도 옵션 제공	호출량 과금
BGE (BAAI)	오픈소스	다국어 변형	자체 호스팅 가능	서빙 인프라 필요
Sentence-Transformers	오픈소스	모델별 상이	실습 채택, 로컬 실행	모델 선택 폭이 넓음

표의 어떤 칸도 최종 답이 아니다. 실습처럼 같은 평가셋으로 두 모델을 직접 측정하는 것이 선택의 근거가 된다.



하이브리드 검색과 재순위

어휘 검색과 의미 검색은 놓치는 질의 유형이 다르다. 두 결과를 합치고, 상위만 다시 정렬한다.

어휘 검색 · BM25

단어가 그대로 일치하는 질의에서 유리하다
희귀어 · 품번 · 코드에서 유리
표현이 다르면 놓친다

의미 검색 · 임베딩

표현이 달라도 의미가 가까우면 찾는다
구어체 · 패러프레이즈에서 유리
정확한 용어 일치에서는 성능이 떨어질 수 있다

결합

순위 병합과 재순위

순위로 합치면 척도가 다른 두 점수를 정규화 없이 결합할 수 있다(RRF).
병합한 상위 N개는 교차 인코더로 다시 정렬한다.

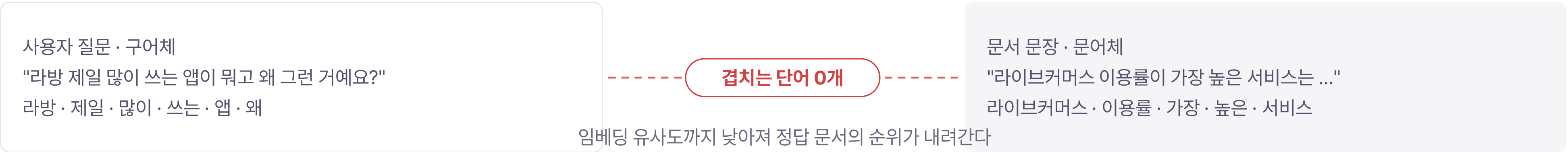
재순위는 상위 N개의 순서를 바꾸는 단계이므로 Recall@N을 올리지 못한다.
올라가는 것은 MRR·NDCG처럼 순위를 반영하는 지표이다.

Query 최적화와 개선 효과 측정



질문과 문서의 어휘 차이

검색 실패의 근본 원인은 질문과 문서의 어휘 차이이다.
해결책은 질문 표현을 바꾸거나, 임베딩 모델을 바꾸는 것이다.



해결책 1 · 질문 표현을 바꾼다

Query Rewriting

"라방 → 라이브커머스", 구어체를 문서의 표현으로 재작성해
질문을 문서에서 사용하는 표현으로 바꾼다
비용: 요청마다 LLM 호출 1회 (자연·비용 증가)

질문 재작성

해결책 2 · 임베딩 모델을 바꾼다

임베딩 모델 교체

도메인 어휘를 잘 반영하는 모델로 바꾸면 질문과 문서가
처음부터 가까운 벡터로 표현된다
비용: 일회성 재인덱싱

모델 교체

두 해결책은 함께 쓸 수 있다
질문 표현을 바꾸는 방법과 임베딩 모델을 바꾸는 방법은 배타적이지 않다. 다만 둘을 함께 적용하면 어느 쪽이 효과를 냈는지 알 수 없으므로, 효과를 분리하는 측정 설계가 따로 필요하다.

사용자의 질문 방식을 바꿀 수는 없으므로, 시스템에서 질문 표현을 바꾸거나 임베딩 모델을 교체한다.



질의 변환 기법 분류

질문을 바꾸는 방법은 여러 가지이고, 비용과 적합한 상황이 서로 다르다.

재작성 (Rewriting)

질문을 문서의 표현으로 다시 쓴다. LLM 호출 1회·검색 1회. 구체체·축약어 질문에 쓴다.

확장 (Expansion)

원 질문에 동의어·관련어를 덧붙인다. 어휘 검색을 함께 쓸 때 효과가 크다.

멀티 쿼리 (Multi-query)

여러 표현으로 바꿔 각각 검색한 뒤 합친다. 검색 횟수가 질의 수만큼 늘어난다.

분해 (Decomposition)

복합 질문을 답하기 쉬운 하위 질문으로 나눈다. 비교·다중 조건 질문에 쓴다.

HyDE

가상의 답변 문서를 만들어 그 문서의 임베딩으로 검색한다. 질문과 문단의 길이 차이를 줄인다.

스텝백 (Step-back)

한 단계 일반화한 질문을 먼저 만들어 함께 검색한다. 지나치게 구체적인 질문에 쓴다.

여섯 가지 가운데 HyDE만 임베딩 대상을 바꾸고, 나머지는 질문 문장을 바꾼다. 선택은 재작성 전후의 검색 지표 비교로 정한다.



Query Rewriting 체인 구현

Rewriting

```
REWRITE = ChatPromptTemplate.from_template(
    "다음 질문을 문서 검색에 적합하게 재작성하세요.\n"
    "- 구어체를 문어체로, 축약어는 정식 명칭으로 "\n"
    "(예: 라방 → 라이브커머스)\n"
    "- 핵심 개념의 동의어를 2~3개 덧붙이되 "\n"
    "의미는 바꾸지 말 것\n"
    "- 재작성한 질문만 한 줄로 출력\n\n"
    "질문: {question}")

rewriter = REWRITE | llm | StrOutputParser()
rewritten = rewriter.invoke(
    {"question": "라방 제일 많이 쓰는 앱이 뭐고 왜 그런 거예요?"})
```

- 재작성 규칙을 프롬프트에 명시한다. 문어체 변환, 정식 명칭, 동의어 확장의 세 가지이다.
- "의미는 바꾸지 말 것"이 핵심 제약이다. 질문이 달라지면 정답도 달라진다.
- 체인은 프롬프트 | 모델 | 파서의 조합이다.

재작성은 검색 전에 질문을 문서의 표현에 맞게 바꾸는 단계이며, 효과는 전후 검색 지표의 차이로 확인한다.



재작성 전후 검색 결과 비교

질문을 문서의 표현에 맞추면 같은 검색기도 다른 결과를 반환한다.

원본: "라방 제일 많이 쓰는 앱이 뭐고 왜 그런 거예요?"

Top-1 · d10-p10 중국 라방 업체 유형	✗
Top-2 · d02-p45 AI 상품추천 솔루션	✗
Top-3 · d01-p11 온라인 쇼핑 관심 정보	✗

정답 d01-p20(주 이용 라이브커머스)이 Top-3에 없음 → MISS

재작성: "라이브커머스 이용률이 가장 높은 서비스"

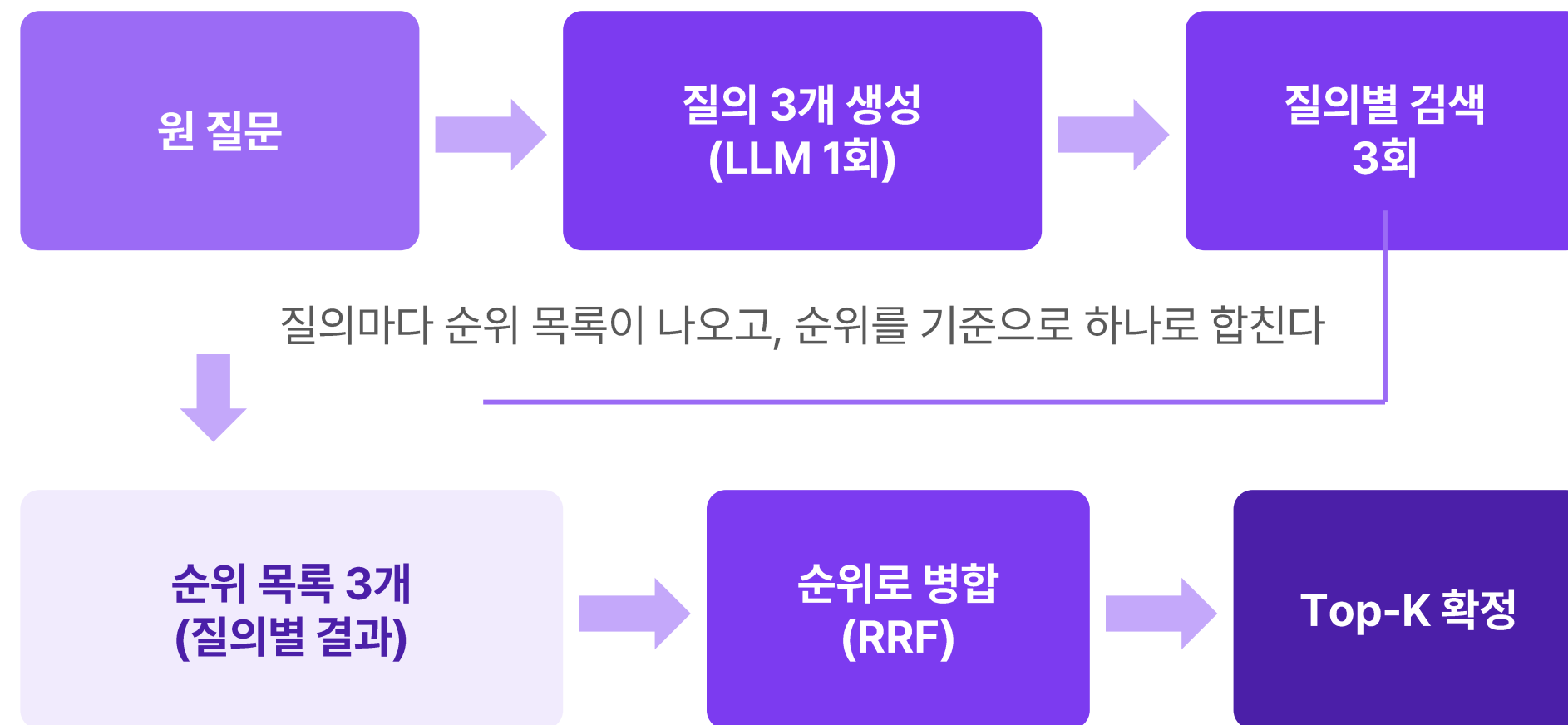
Top-1 · d01-p20 주 이용 라이브커머스	✓
Top-2 · d07-p02 라이브커머스 이용자 특성	✗
Top-3 · d01-p11 온라인 쇼핑 관심 정보	✗

정답이 1위로 → Recall@3 회복, RR = 1.0

질문과 문서에 겹치는 어휘가 생기면서 임베딩 유사도가 올라간다. 결과는 예시이며 실행마다 순위는 달라질 수 있다.



멀티 쿼리와 결과 병합



- 질의를 여러 개로 늘리면 표현 하나에 의존하지 않게 되어 회수가 올라간다. 대신 검색 횟수가 질의 수만큼 늘어나 지연과 비용이 함께 커진다.
- 합칠 때는 점수가 아니라 순위를 쓴다. 문서마다 각 목록의 $1/(60 + \text{순위})$ 값을 합산하는 방식이 널리 쓰이며, 척도가 다른 점수를 정규화하지 않아도 된다.
- 평가할 때는 문서 ID로 중복을 먼저 제거한다. 같은 근거가 여러 질의에서 중복으로 검색되면 지표가 실제보다 높게 측정된다.



재작성이 실패하는 경우

재작성이 항상 이득인 것은 아니다. 이미 잘 검색되는 질문에서는 점수가 내려갈 수 있다.

의 의미 변형

질문의 조건이나 범위가 바뀌어
다른 질문이 된다

확 과도한 확장

관련어를 과도하게 더해 핵심 용어의
비중이 낮아진다

명 고유명사 훼손

제품명·문서 번호·코드가
일반어로 바뀐다

방어 방법

- 의미를 바꾸지 말라는 제약을 프롬프트에 명시하고, 재작성 결과를 원문과 나란히 출력해 확인한다
- 원 질문과 재작성 질문을 함께 검색해 순위로 합치면 재작성이 해로운 문항의 손실을 줄일 수 있다
- 전체 평균과 함께 문항별로 전후를 비교한다. 점수가 올라간 문항과 떨어진 문항의 수가 판단 근거이다

적용 여부는 문항별 비교로 판단한다. 고유명사 훼손이 잦으면 질문 재작성만으로는 한계가 있다.



전문용어가 밀집한 도메인에서는 범용 임베딩이 유사도를 제대로 계산하지 못할 수 있다.

의 의료

질환·약어·성분명이 일반
어휘와 의미 공간이 다르다

법 법률

판례·조문 특유의 문형과
한자어가 많다

금 금융

상품·규제 용어와 수치
표현이 밀집해 있다

특화 모델 도입 판단

- 도메인 코퍼스로 학습된 특화 모델이 도입 후보가 된다
- 전문용어 질문을 분리한 평가셋을 만들고, 일반 질문과 전문 질문의 성능을 나누어 측정한다
- 성능을 나누어 보면 특화 모델의 효과가 어디에서 발생하는지 드러난다

특화 모델 도입 여부도 감각이 아니라 측정으로 정한다. 실습 5에서 RUN_KURE를 켜면 같은 절차로 한국어 특화 모델을 측정할 수 있다.

도메인 특화 모델은 학습 코퍼스와 제공 형태부터 확인하고 후보를 좁힌다.

도메인	모델	학습 코퍼스	다국어
의료	NeuML/pubmedbert-base-embeddings	PubMed 논문 제목·초록	영어 전용
법률	voyage-law-2	법률 문서 (장문 최적화)	다국어 지원
금융	voyage-finance-2	금융 도메인 문서	다국어 지원
한국어	nlpai-lab/KURE-v1	한국어 검색 쌍 약 200만 건	한국어·영어

학습 코퍼스가 자체 문서와 겹치는지, 다국어를 지원하는지, 자체 호스팅이 가능한지를 함께 본다. 표의 어떤 칸도 도입 근거가 아니다.

실습의 CANDIDATES는 OpenAI 임베딩 두 가지와 로컬 MiniLM이며,
RUN_KURE=True로 두면 KURE-v1(2.3GB)을 같은 절차로 비교한다.

Fine-tuning과 공개 특화 모델 도입은 기대 성능과 부담해야 할 비용이 다르다.

Fine-tuning · 직접 학습

기대

자체 데이터에 맞춘 높은 성능을
기대할 수 있다

부담

학습 데이터 구축·학습 비용·유지보수를
직접 부담한다

공개 특화 모델 도입

기대

도입 비용이 낮고 검증이 빠르다
교체 실험으로 확인한다

부담

대상 도메인과 어긋나면 기대만큼
오르지 않을 수 있다

순서가 중요하다. 먼저 공개 특화 모델을 평가셋으로 측정하고, 그래도 부족할 때 Fine-tuning을 검토한다.



2×2 효과 분리 실험

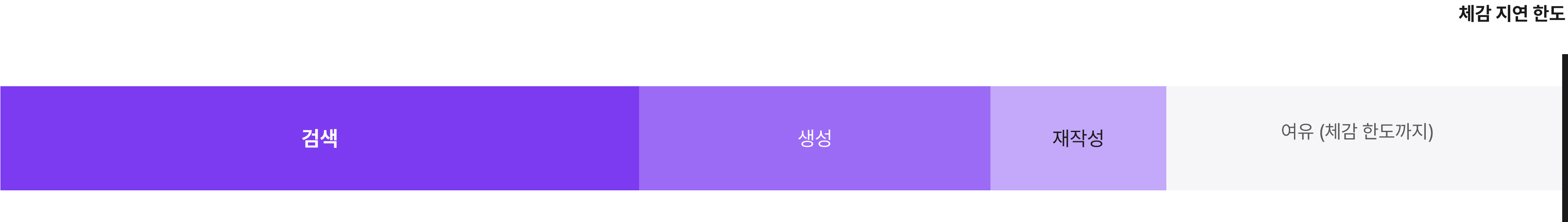
모델 A/B × 원본/재작성 네 조합을 같은 평가셋으로 측정하면 두 해결책의 단독 효과가 분리된다.

	원본 질문	재작성 질문
모델 B (교체)	모델 B × 원본 교체 단독 효과 기준선과의 차이가 모델 교체의 순수 효과이다	모델 B × 재작성 결합 효과 확인 두 처방을 함께 적용했을 때의 효과를 확인한다
모델 A (기준)	모델 A × 원본 기준선 현재 시스템의 성능이며, 나머지 세 조합을 비교하는 기준점이다	모델 A × 재작성 재작성 단독 효과 기준선과의 차이가 질의 재작성의 순수 효과이다

두 해결책의 효과가 비슷하면 일회성 비용인 모델 교체를 먼저 검토한다.



질의 재작성은 요청마다 LLM 호출을 한 번 추가한다.
전체 지연과 비용에서 차지하는 비중을 확인한다.



대안 비교: 재작성은 질문 수에 비례하는 비용, 모델 교체는 재인덱싱 한 번의 일회성 비용이다.

재작성이 응답 시간 예산의 한 구간을 차지한다.

모든 질문에 활성화할지, 실패가 잦은 질문에만 활성화할지 선택하는 지점이 생긴다.



메타데이터 필터와 질의 라우팅

값이 정해진 조건은 필터로 먼저 좁히고, 질문 유형에 따라 검색 경로를 나눈다.

검색 범위 좁히기

```
retriever = vectordb.as_retriever(  
    search_kwargs={  
        "k": 3,  
        "filter": {"$and": [  
            # 라이브커머스 문서  
            {"doc_id": {"$in": ["d01", "d07", "d10"]}},  
            {"year": {"$gte": 2021}},  
        ]},  
    })  
  
route = classify(question) # 시장 동향 / 솔루션 / 범위 밖  
if route == "범위 밖":  
    return "문서에서 확인할 수 없습니다"
```

- 메타데이터 필터는 검색 대상을 먼저 줄인다. 문서 ID·발행 연도·문서 종류처럼 값이 정해진 조건에 쓴다.
- 필터는 청크에 메타데이터가 남아 있어야 적용된다. 인덱싱 단계의 검증이 여기서 쓰인다.
- 라우팅은 질문 유형에 따라 검색 경로를 나눈다. 범위 밖 질문을 미리 걸러 내면 환각을 줄일 수 있다.
- 평가에서는 문항마다 필터 조건과 경로를 고정해 둔다. 조건이 달라지면 서로 다른 대상을 비교하게 된다.

필터는 LLM 호출 없이 검색 범위를 줄이는 수단이다.
재작성보다 비용이 낮으므로 먼저 검토한다.



"어디가 잘못되었는가?" 이제 숫자로 답할 수 있다.

Motivation & Challenges

왜 필요하고 왜 어려운가

- 평가가 필요한 이유 · 현황 파악과 개선 검증
- 직렬 결합과 성능 상한 · R·A·G 단계 분해
- 실패 세 유형 분류 · 단계별 지표 대응표

Retrieval Evaluation

근거를 제대로 찾는가

- Hit·Precision·Recall·MRR·NDCG 구현
- 청킹 설정 선택과 청크 품질 검사
- 임베딩 모델 비교와 색인 설정
- 재작성·특화 모델의 효과를 2×2로 분리

오늘 확보한 것

실패 유형 분류 기준

인덱스 검증 절차

Recall·MRR·NDCG 기준선



데모와 프로덕션의 차이

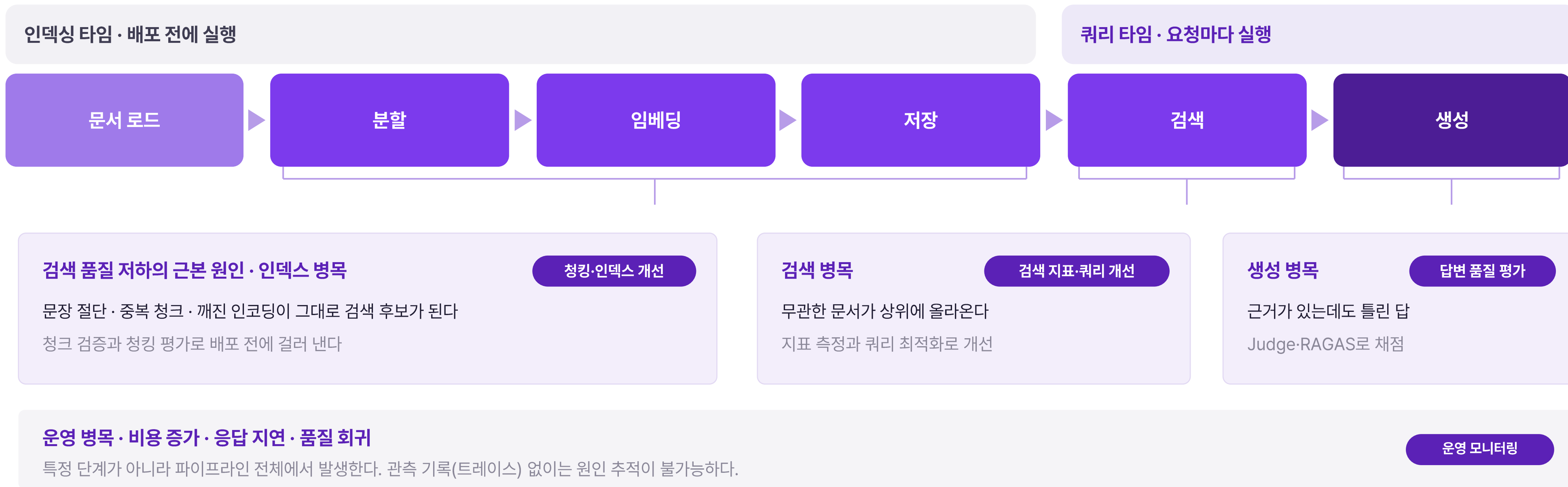
준비된 질문 10개에만 잘 동작하는 데모와, 어떤 질문이든 들어오는 프로덕션은 다른 시스템이다.

데모	VS	프로덕션
고른 질문 10개 만든 사람이 아는 질문만		오타·구어체·범위 밖 질문 질문 다양성이 크게 증가
문서 100건 · 고정 인덱스 품질이 일정하게 유지됨		문서 수만 건 · 계속 갱신 인덱스 품질이 서서히 저하
트래픽 소량 · 실패 재현 용이 즉시 다시 질의해 확인 가능		"가끔 이상한 답" 제보 기록이 없으면 재현 불가

각각 검색 품질 저하, 응답 일관성 부족, 원인 추적 불가 문제로 이어진다.

프로덕션 RAG에서 자주 발생하는 문제

파이프라인 전체를 한 장에 놓고, 문제가 생기는 위치와 이를 다루는 챕터를 함께 정리했다.



프로덕션 세 가지 문제와의 대응 관계



사용자에게는 어떤 문제든 "답이 이상하다"라는 하나의 증상으로 나타난다. 그래서 파이프라인 위에서 문제의 위치를 먼저 특정한 뒤, 채점과 운영 영역으로 범위를 넓혀 간다.



RAG 평가가 어려운 여섯 가지 이유

여섯 가지 특성 때문에 문자열 일치 채점이 아닌 별도의 평가 설계가 필요하다.

단일 정답이 없다

같은 질문에 표현이 다른 정답이 여럿이다. 문자열 일치로는 채점할 수 없다.

오류가 중첩된다

검색 오류 위에 생성 오류가 쌓인다. 최종 답만 보면 어느 단계의 잘못인지 알 수 없다.

정확성 ≠ 근거성

사실이지만 문서에 없는 답, 문서에 충실하지만 틀린 답을 구분해야 한다.

정답지 없이 평가

reference-free 평가는 답의 내부 정합성으로 채점하므로 기준 설계가 어렵다.

Judge도 일관되지 않다

LLM 채점자는 위치·상황함 편향이 있고 점수도 불안정하다.

비결정성

같은 입력에도 출력이 달라져 단건 비교가 아니라 분포와 추이로 판단해야 한다.



실패 판정 트리와 처방

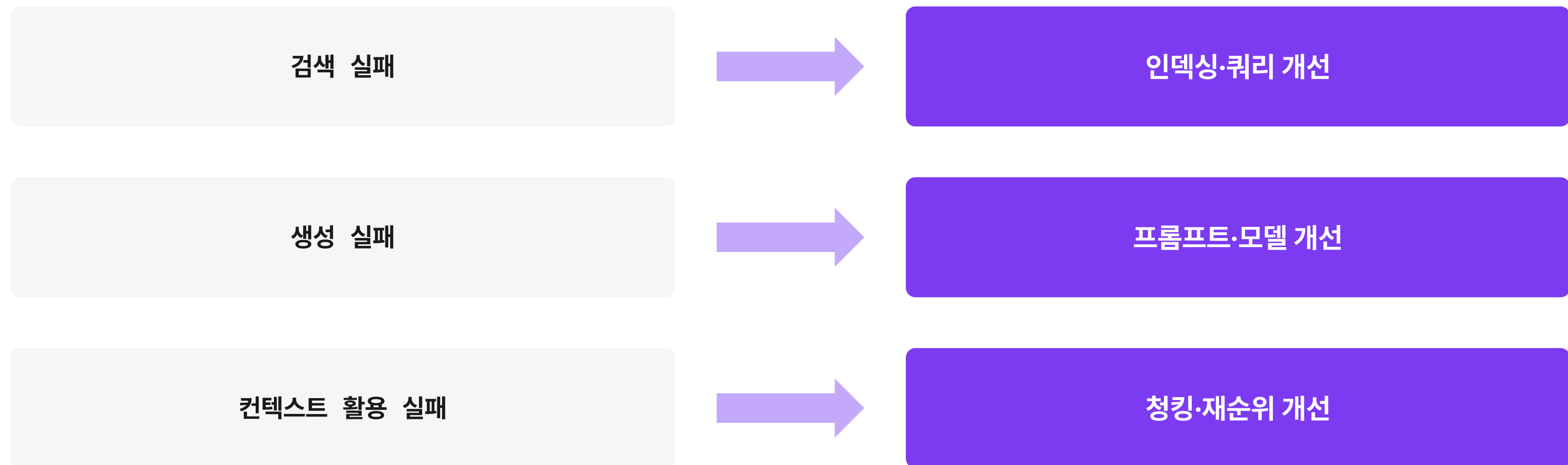
실행 기록에서 두 가지만 확인하면 실패 유형이 구분되고, 유형에 따라 조치가 정해진다.



두 가지를 판정하려면 요청마다 검색된 컨텍스트와 답변이 남아 있어야 한다.



유형을 알면 조치가 정해진다. 유형을 모르면 개선 지점을 특정할 수 없다.



조치가 항상 효과를 내는 것은 아니다. 근거 자체가 없으면 프롬프트를 아무리 강화해도 정답이 나올 수 없다



LangFuse: LLM 관측 플랫폼



- LangFuse는 오픈소스 LLM 관측 플랫폼이다. 요청마다 프롬프트·응답·토큰·지연·비용과 중간 단계(검색·도구 호출)를 구조화된 트레이스로 기록한다.
- 기록 위에서 대시보드·스코어링·LLM-as-a-Judge 평가·프롬프트 관리까지 이어진다. 이번 과정 실습의 관측 도구로 사용한다.



Trace 안에 단계(Observation)가 계층으로 쌓인다

RAG 한 번의 질문이 하나의 Trace가 되고, 그 안에 retrieval·generation 단계가 중첩된다

Langfuse v3.82.0

Langfuse Demo / langfuse-docs / Traces

Go to... K

Home

Dashboards

Observability

Tracing

Sessions

Users

Prompt Management

Prompts

Playground

Evaluation

Scores

LLM-as-a-Judge

Human Annotation

Datasets

Settings

Support

FK Felix Krauth felix@langfuse.com

Trace: eb346ba8-19d9-4368-9b71-022cd17f25d7 with-context ☆

qa ID

2025-07-16 23:44:16.449

Session: lf.docs.conversation.s7A0W7i User ID: u-M9QTUzj Env: default Latency: 11.58s Total Cost: \$0.000371

1,313 → 291 (Σ 1,604) Release: 044d0162fe3dbddd09f88a554521ddf73726618c

Preview Scores Formatted JSON

Input

"How to update the input and output in langgraph traces?"

Output

"To update the input and output in Langfuse traces, it's essential to understand that Langfuse allows meticulous tracking of steps within the AI process by logging both inputs and outputs of large language models (LLMs). This means that every interaction you have can be captured and displayed transparently within the UI.

Here's how you can effectively manage and visualize these updates:

Steps to Update Traces

1. **Log Inputs and Outputs**: Ensure you're logging the inputs and outputs of

...expand (989 more characters)
"

Metadata

{
 pathname: "/docs/demo"
}

qa

11.58s \$0.000371

retrieval

1.55s Σ \$0.000001

prompt-embedding DEBUG

0.67s 12 → 0 (Σ 12) \$0.000001

vector-store

0.69s

context-encoding

fetch-prompt-from-langfuse

0.47s

generation

9.57s 1301 → 291 (Σ 1592) \$0.00037

1

실패 재현

단계마다 입력·출력·소요 시간이 그대로 기록되어 실패를 재현할 수 있다

2

비용 집계

토큰 수와 비용, 전체 지연(Latency)이 요청 단위로 집계된다

3

단위 확장

Session·User ID로 묶으면 대화·사용자 단위 분석으로 확장된다

트레이스 스키마: trace·span·generation

observation은 span·generation·event 세 종류이고, 계층마다 남는 필드가 다르다.

```
trace
  input · output · metadata
  tags · user_id · session_id · version

+-- span          구간이 있는 단계
|   input · output · 시작 · 종료 시각
|
+-- generation    LLM 호출
|   model · usage · cost · 첫 토큰 시각
|
+-- event         단일 시점 기록
```

span

구간 기록

시작과 끝이 있는 단계

검색이나 도구 호출처럼 구간을 갖는 작업을 기록한다. 입력과 출력, 소요 시간이 남는다.

generation

LLM 호출

모델 호출 전용 기록

span의 필드에 더해 모델 이름, 토큰 사용량, 비용, 첫 토큰 시각이 남는다. 비용·지연 집계는 입력이 된다.

재현 1 무엇을 남기는가

질문·검색 청크·프롬프트·답변을 남기면 같은 요청을 다시 구성할 수 있다.

재현 2 무엇으로 찾는가

user_id·session_id·version·tags를 남기면 제보된 요청을 사용자·기간·버전으로 좁혀 찾을 수 있다.

generation에 남는 토큰 사용량과 비용은 이후 시스템 성능 집계는 입력이 된다.



실습 03 · LangFuse 연동

```
from langfuse import get_client
from langfuse.langchain import CallbackHandler

handler = CallbackHandler()
callbacks = []
if get_client().auth_check():
    callbacks = [handler]

chain = NAIVE_PROMPT | llm | StrOutputParser()
answer = chain.invoke(
    {"context": ctx, "question": q},
    config={"callbacks": callbacks},
)
```

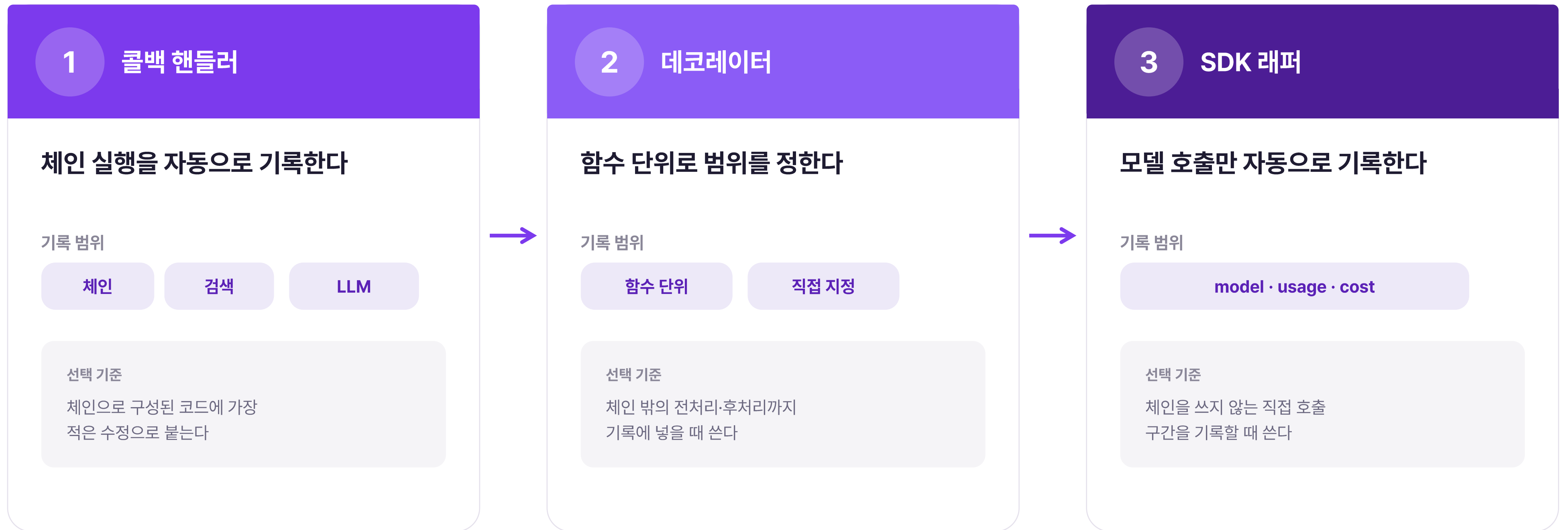
- CallbackHandler 하나로 체인·LLM·리트리버 호출이 모두 기록된다. (langfuse 3.x)
- auth_check()로 키를 확인하고, 키가 없으면 콜백 없이 진행한다.
- invoke의 config에 callbacks를 넘기는 것이 유일한 연결 지점이다.

**관측은 코드 수정 없이 콜백으로 덧붙인다.
기록이 있어야 실패를 재현할 수 있다.**



계측 방법 세 가지

무엇을 어디까지 기록할지에 따라 계측 방법을 고른다.

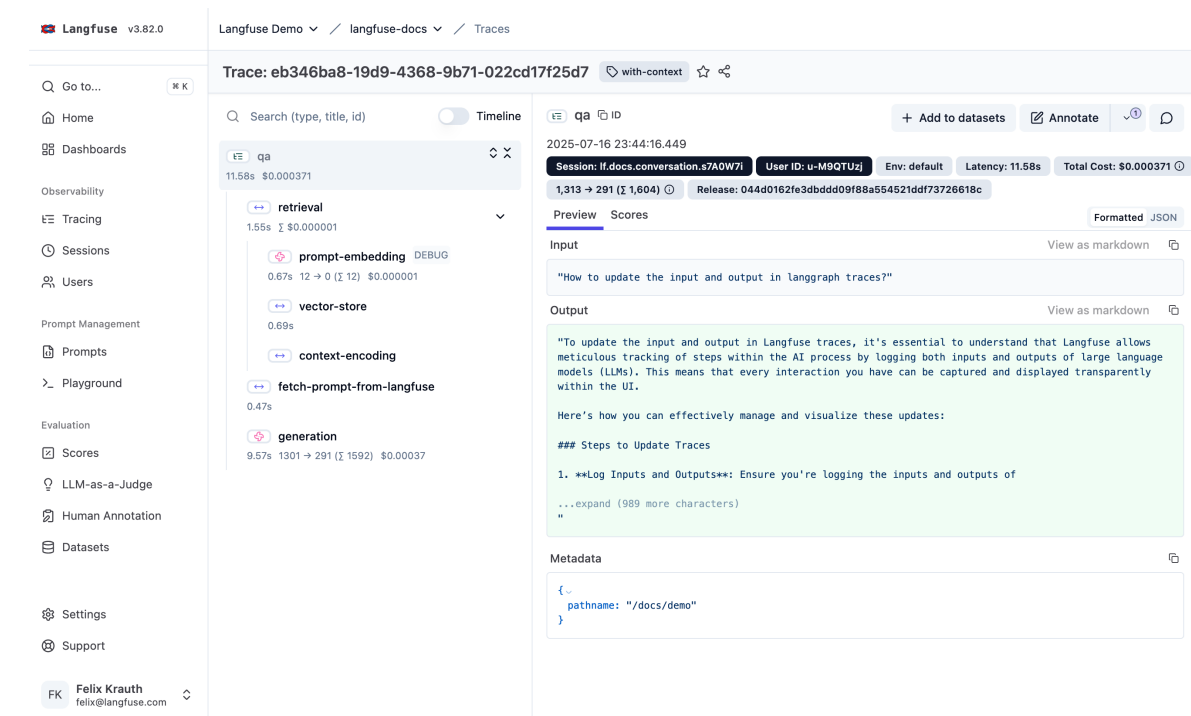


세 방법은 함께 쓸 수 있다. 데코레이터로 묶은 함수 안에서 콜백 핸들러와 래퍼가 각각 하위 기록을 남기는 구성이 흔하다.



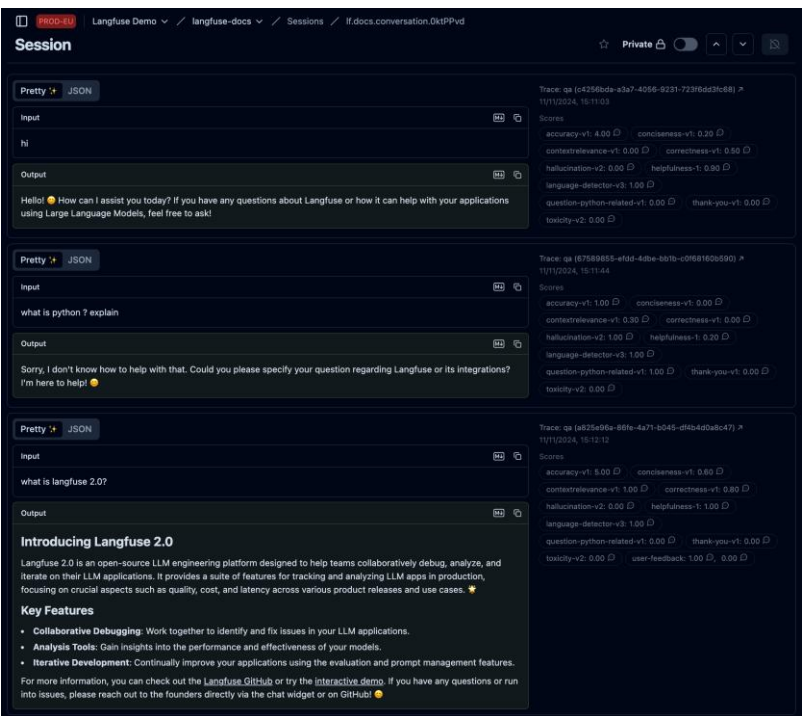
LangFuse 대시보드 세 화면

LangFuse는 요청·대화·지표의 세 화면으로 실패를 재현하고 추이를 관찰한다.



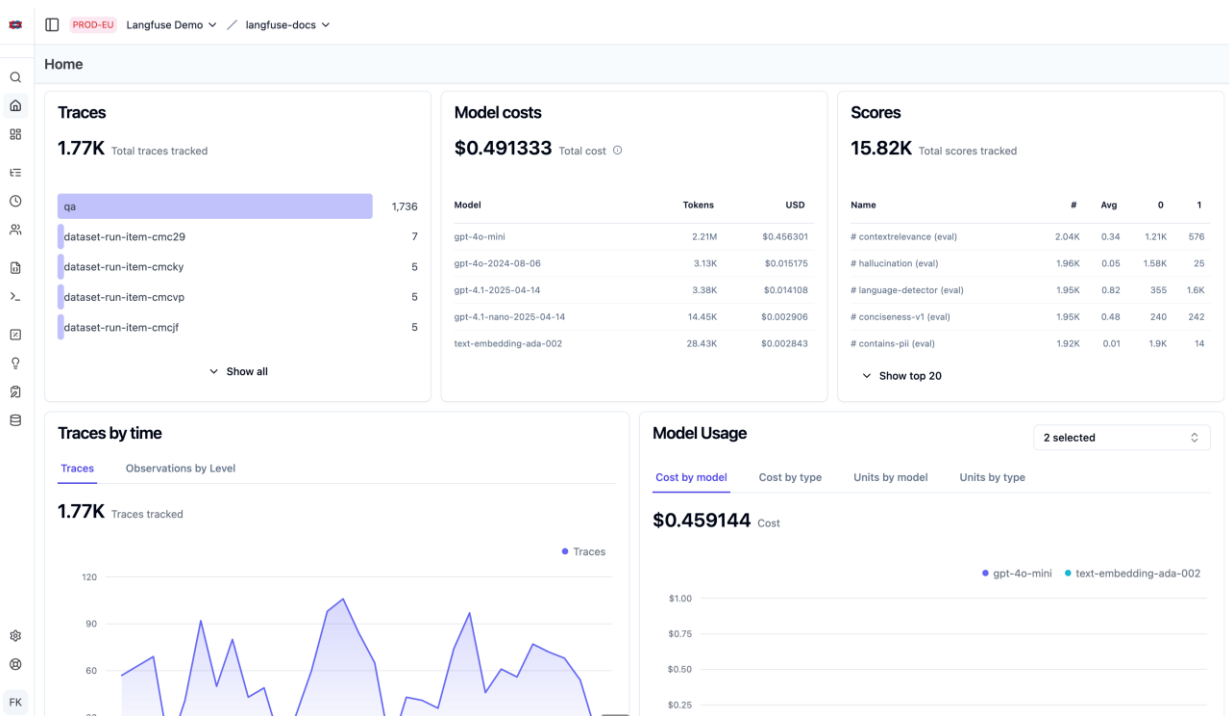
Traces: 요청 단위 기록

실패 제보가 오면 해당 트레이스를 열어
검색 결과와 답변을 그대로 재현한다.



Sessions: 대화 단위 묶음

멀티턴 대화를 세션으로 묶어 사용자 흐름 속에서
실패 맥락을 본다.



Dashboards: 지표 집계

비용·지연·불륨·품질 점수를 모델·사용자·버전별로 나누어 확인한다.



정확성 vs 근거성

정확성은 실제 사실과 비교하고, 근거성은 제공된 컨텍스트와 비교한다.

정확성 (Accuracy)	근거성 (Faithfulness)
- 실제 사실과 맞는 답인가를 본다 - 예: "연차는 15일", 사실이면 통과 - 정답지가 있어야 채점할 수 있다 채점에 정답지 필요	- 제공된 컨텍스트에 충실한 답인가를 본다 - 예: 문서에 없는 15일을 말하면 실패 - 정답지 없이도 채점할 수 있다 정답지 없이 채점 가능

정답지 없이도 채점할 수 있는 근거성이 RAG 평가의 기본 기준이 된다.

reference-based

- 사람이 만든 정답(ground truth)과 비교해 채점한다.
- Context Recall처럼 정답 문서·정답 답변이 필요하다.

reference-free

- 정답 없이 질문·컨텍스트·답 사이의 정합성으로 채점한다.
- Faithfulness처럼 새 질문에도 바로 적용할 수 있다.

정답이 없어도 평가는 가능하다.
대신 무엇을 기준으로 채점하는지가 지표마다 다르다.



평가셋 규모와 결과의 신뢰도

문항이 적으면 같은 점수라도 폭이 넓다. 폭을 알고 나서야 차이를 해석할 수 있다.

1 문항이 적으면 폭이 넓다

10문항에서 8개 통과면 80%이지만, 95% 신뢰구간은 약 49%~94%이다.

이 폭 안에서는 60%인 시스템과 90%인 시스템을 구분할 수 없다.

실습 평가셋



5~10문항

2 문항을 늘리면 폭이 좁아진다

같은 80%에서 30문항이면 약 63%~91%, 100문항이면 약 71%~87%이다.

폭은 문항 수의 제곱근에 반비례해 줄어든다. 문항을 네 배로 늘리면 폭이 절반이 된다.

10

30

100

300

문항 수

3 단건 차이로 결론 내지 않는다

두 설정의 차이가 폭 안에 들어가면 개선으로 보기 어렵다.

같은 문항으로 짝지어 비교하면 같은 문항 수에서도 더 작은 차이를 판별할 수 있다.

폭 $\propto 1/\sqrt{n}$

판별력 상승

실습 평가셋은 구조를 익히기 위한 규모이다. 운영 판정에는 문항을 늘리고 통계 검정을 함께 적용한다.

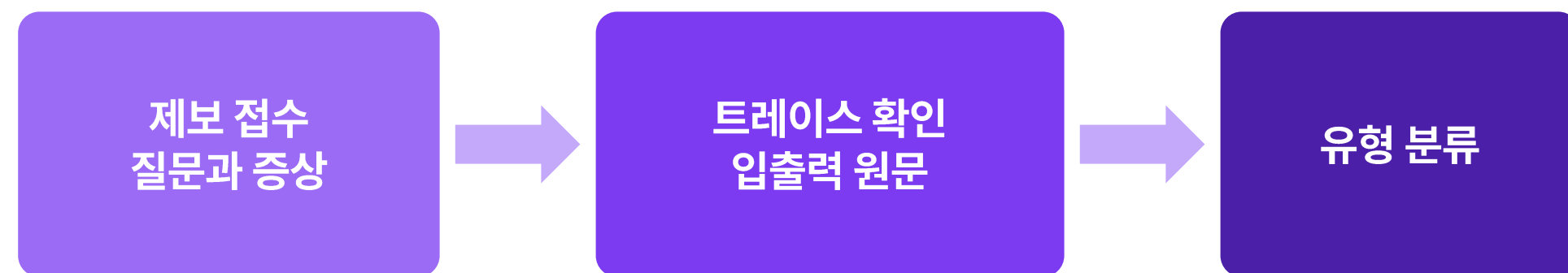
Naive RAG의 실패 지점과 LangFuse



실패 사례에서 개선 백로그까지

제보가 조치로 이어지려면 다섯 단계를 거친다. 기록이 없으면 두 번째 단계에서 끊긴다.

기록 조회

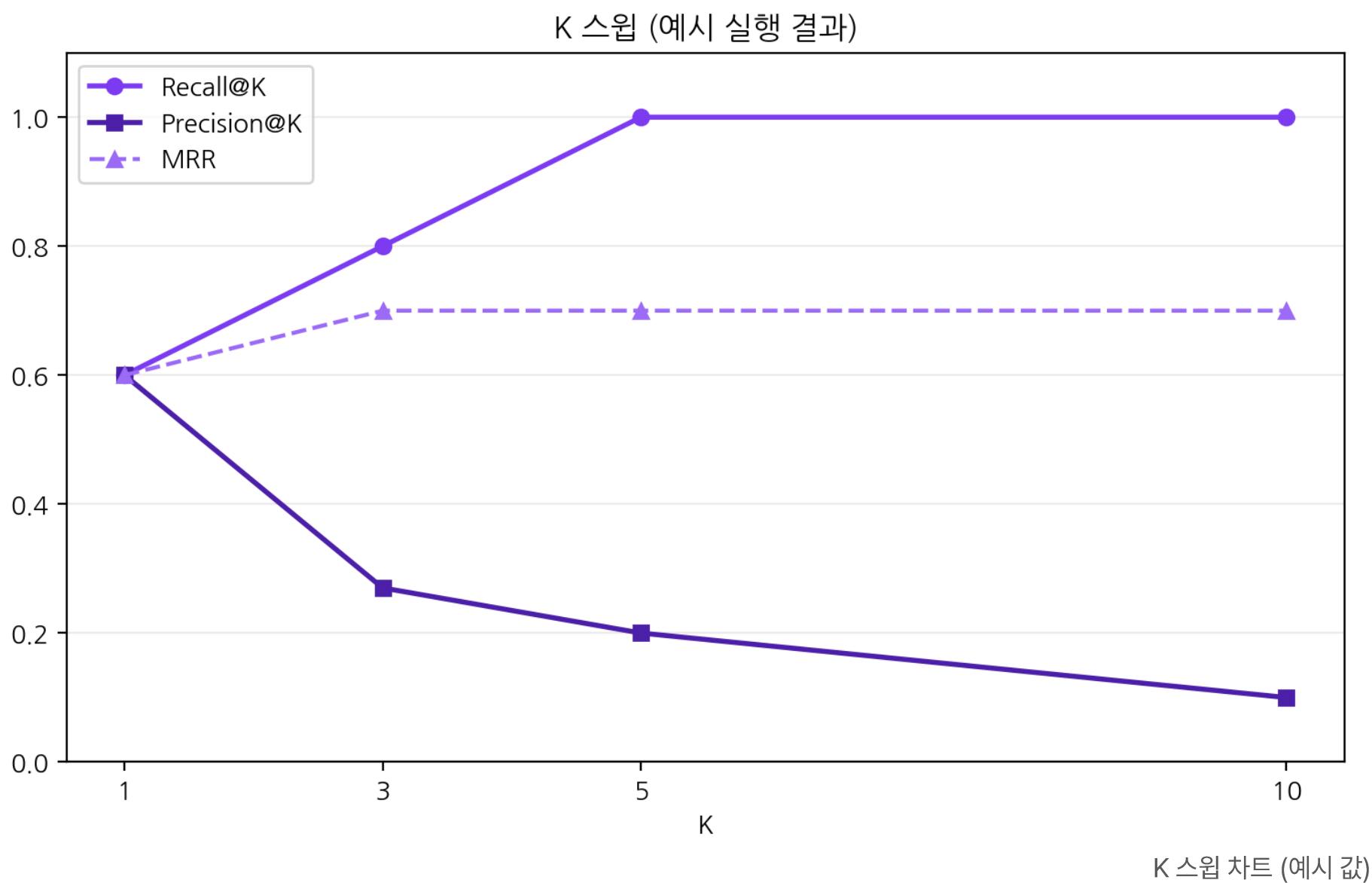


빈도 집계 → 개선 백로그 항목화

- 1) 제보 접수: 질문과 증상을 구체적으로 기록한다.
- 2) 트레이스 확인: 검색 청크·프롬프트·답변 원문을 본다.
- 3) 유형 분류: 검색·생성·컨텍스트 활용 실패 중 하나로 나눈다.
- 4) 빈도 집계: 유형별 건수로 우선순위를 정한다.
- 5) 백로그 항목화: 유형·건수·조치·확인할 지표를 남긴다.



K 스왑과 트레이드오프



K를 키우면 Recall은 상승하고 Precision은 하락한다.

같은 평가셋에서 K를 1, 3, 5, 10으로 바꿔 가며 지표를 측정한 결과이다.

Recall은 K가 커질 때 줄어들지 않는다. 후보를 늘리면 회수는 늘어난다.
대신 Precision이 떨어지고, 컨텍스트가 길어져 토큰 비용·지연이 늘어난다.
K는 지표가 아니라 요구사항에 따라 결정된다. 품질·비용·지연의 절충점을 선택한다.

K는 지표만으로 정하기 어렵다. 품질·토큰·지연 세 축의 상한을 함께 확인한 뒤 정하는 것을 권장한다.

확인 축	확인 방법	K에 주는 상한
품질	K를 1·3·5·10으로 바꿔 Recall@K·Precision@K를 측정한다	Recall이 더 오르지 않는 지점
토큰 예산	$K \times \text{평균 청크 길이}$ 로 컨텍스트 토큰을 계산한다	프롬프트 예산에서 역산한 K
지연	K를 키우며 검색·생성 소요 시간을 측정한다	체감 한도를 넘기지 않는 K
종합	세 상한 가운데 가장 작은 값을 K로 두는 것을 권장한다	요구사항이 바뀌면 다시 정한다

품질만 보고 K를 키우면 토큰과 지연 측면의 부담이 커질 수 있다. 세 상한 가운데 가장 작은 값을 K로 두는 것을 권장한다.